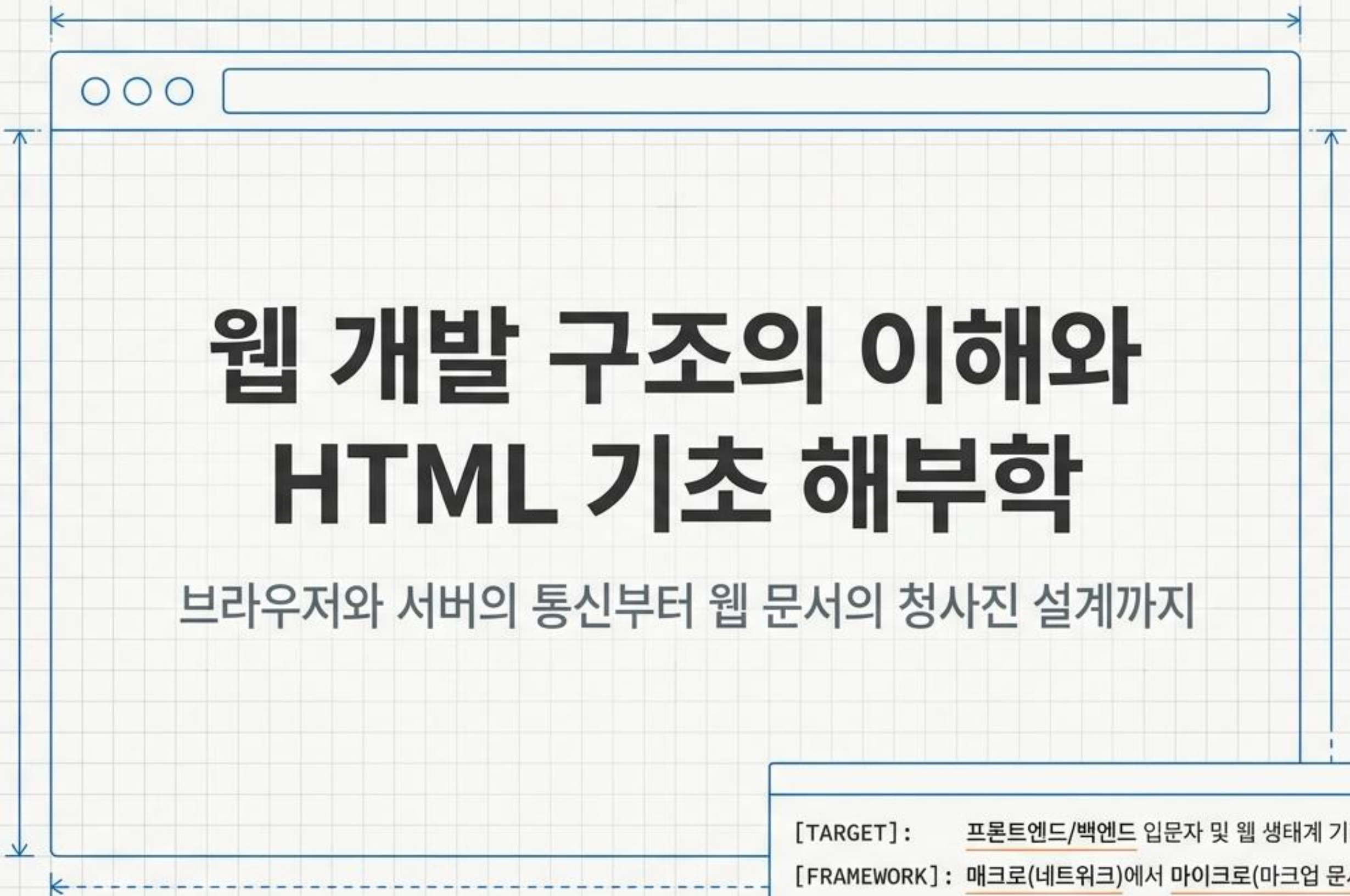




웹 개발 구조의 이해와 HTML 기초 해부학

브라우저와 서버의 통신부터 웹 문서의 청사진 설계까지

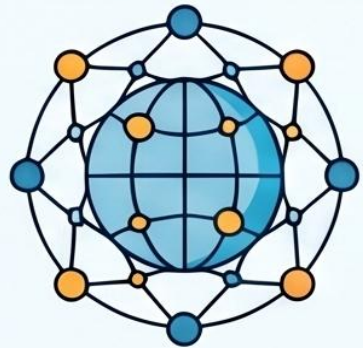
[TARGET] : 프론트엔드/백엔드 입문자 및 웹 생태계 기획자

[FRAMEWORK] : 매크로(네트워크)에서 마이크로(마크업 문서)로의 구조적 접근

웹 개발의 시작: HTML과 기초 개념 가이드

🌐 웹과 HTML의 핵심 개념

인터넷 vs. 웹, 서로 다른 개념입니다.



인터넷

인터넷은 전 세계를 연결하는 거대한 통신망이며, 웹은 그 안에서 정보를 공유하는 서비스입니다.



웹 페이지의 뼈대, HTML

하이퍼텍스트와 마크업 언어를 결합해 콘텐츠의 구조와 연결성을 정의합니다.

브라우저의 '렌더링' 과정

클라이언트(브라우저)가 서버로부터 받은 HTML 코드를 해석하여 시각적인 화면으로 표시합니다.



클라이언트(브라우저)



서버



클라이언트(브라우저)



HTML 코드 해석



시각적인 화면

웹 개발 프로세스에 참여하는 주요 직무별 역할 정의



	주요 역할	주요 구야	사용 도구/언어
 웹 디자인	시각적 표현 및 UX 기획	시각적 표현 및 UX 기획	그래픽 도구, 웹 코딩
 프론트엔드	사용자 화면 및 상호작용 구현	사용자 화면 및 상호 구현	HTML, CSS, JavaScript
 백엔드	서버 동작 및 데이터 제공 정의	서버 동작 및 데이터 제공 정의	Java, Python, DB 관리

💻 개발 환경 구축 및 실습



필수 설치 도구: 크롬 & VS Code

표준 기술 지원이 우수한 크롬 브라우저와 무료 코드 에디터인 VS Code를 설치합니다.



.html 파일 생성과 실행

VS Code에서 확장자 .html로 파일을 저장한 후 브라우저로 열어 결과를 확인합니다.

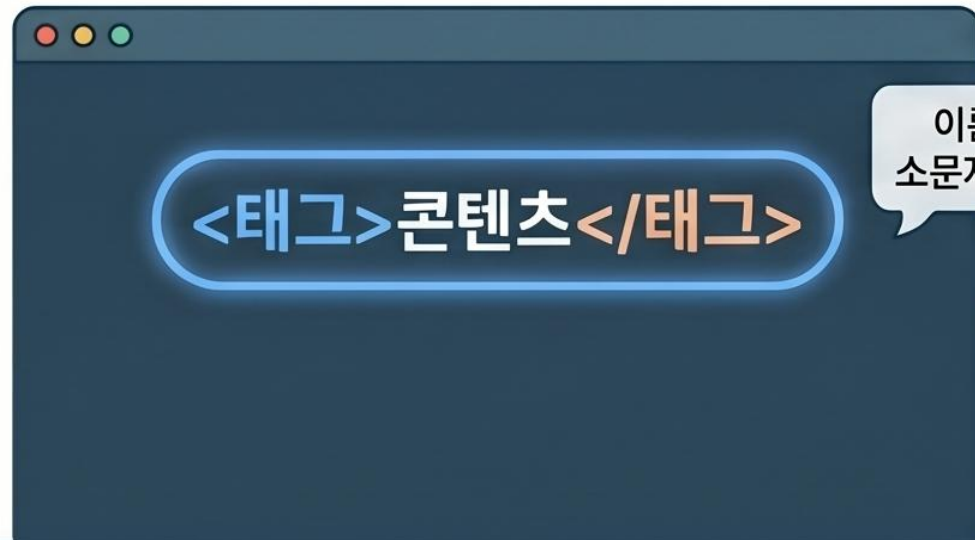


HTML5, 현대 웹의 표준

팀 버너스리가 제안한 HTML의 최신 버전으로, 오늘날 대부분의 브라우저가 지원합니다.

HTML 입문 가이드: 웹 페이지의 기본 뼈대와 문법

HTML 기본 문법과 요소 유형



블록(Block) vs 인라인(Inline) 요소

블록
(Block)

가로 전체 차지

인라인
(Inline)

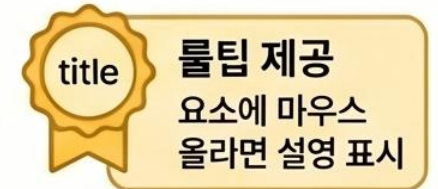
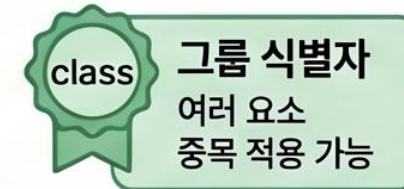
콘텐츠 크기만큼만 공간 차지

속성(Attribute)을 통한 기능 확장

`<태그 속성명="속성값">`



부가 기능 추가



표준 문서 구조와 핵심 태그

HTML5 표준 문서의 3단계 구조

문서의 메타 정보와 제목 설정

`<meta charset="UTF-8">`

인코딩 설정

`<title>웹 페이지 제목</title>`

브라우저 탭 제목 표시

DOCTYPE 선언

정보 설정 (head)

실제 콘텐츠 (body)

콘텐츠 레이아웃을 위한 컨테이너

`<div>` (블록)
영역 묶어 관리

`<span>` (인라인)
텍스트 일부분 묶기

웹 개발의 시각적 완성: CSS 핵심 요약 가이드

선택자(Selector) - 디자인할 대상 정하기

기본 선택자 (ID vs Class)



#ID
중복 없는 고유 요소에는 #ID를,
여러 요소 공통 적용에는 .Class를 사용합니다.

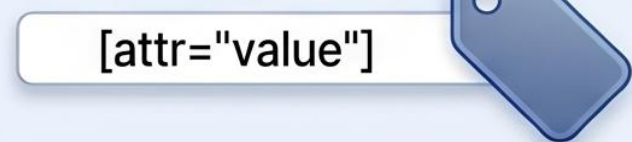


구조 및 가상 선택자

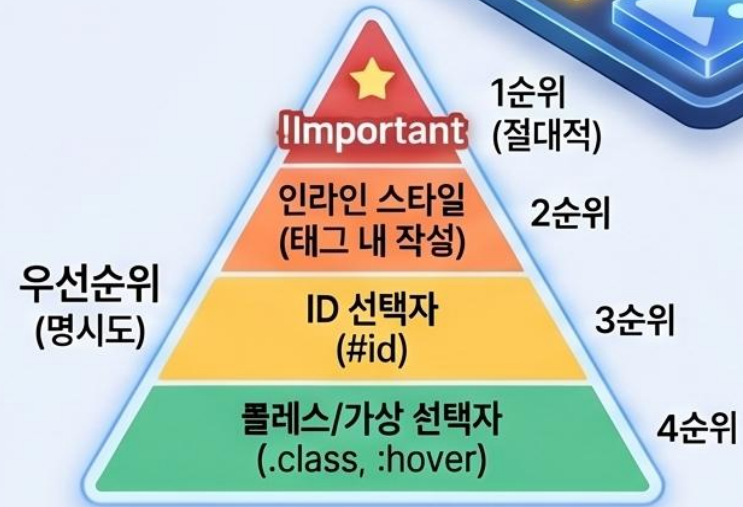
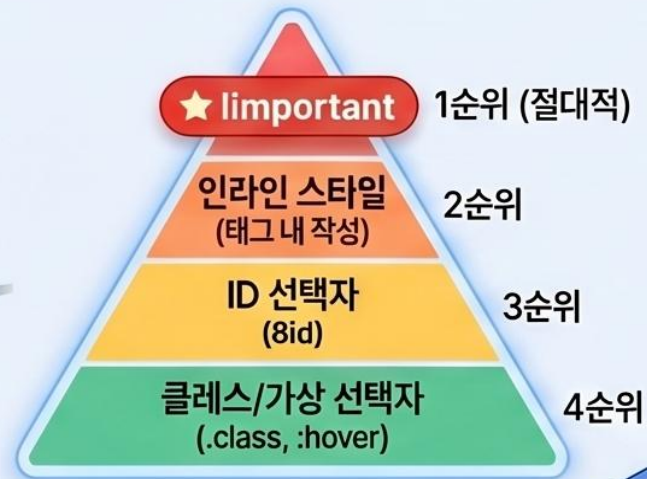


:nth-child(순서 제어) 등으로 역등적인
디자인이 가능합니다.

속성 선택자 활용

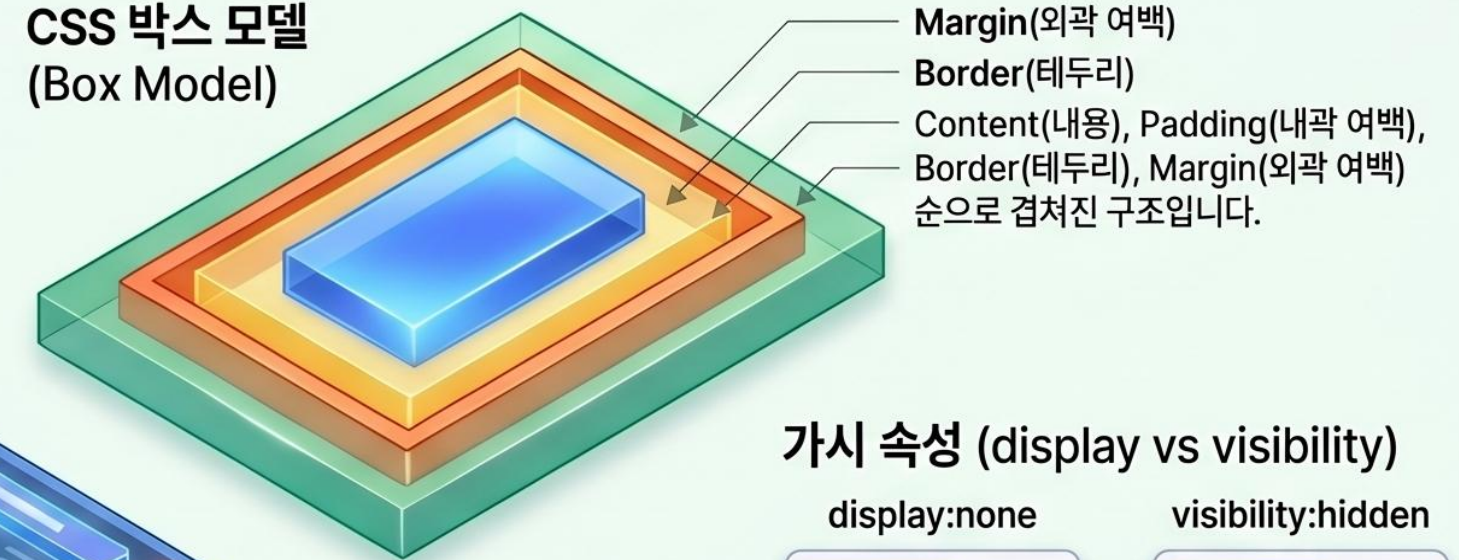


[속성*=값] 차함 태그의 특정 속성값을 찾아
정밀하게 스타일을 적용합니다.



박스 모델과 레이아웃 - 형태와 배치 제어

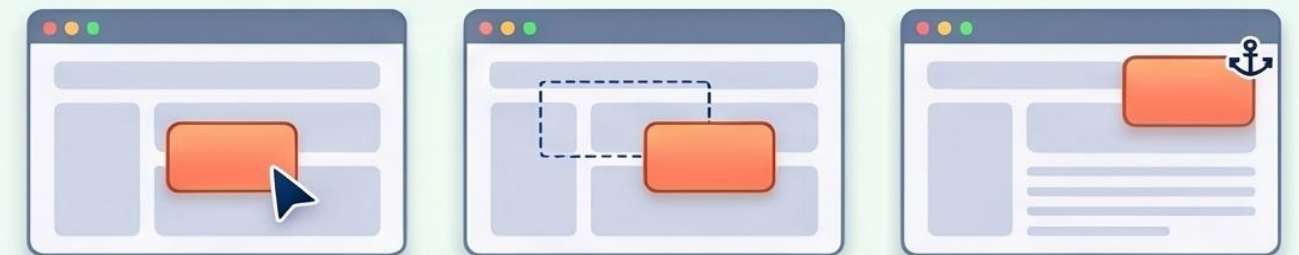
CSS 박스 모델 (Box Model)



가시 속성 (display vs visibility)



위치 속성 (Position)

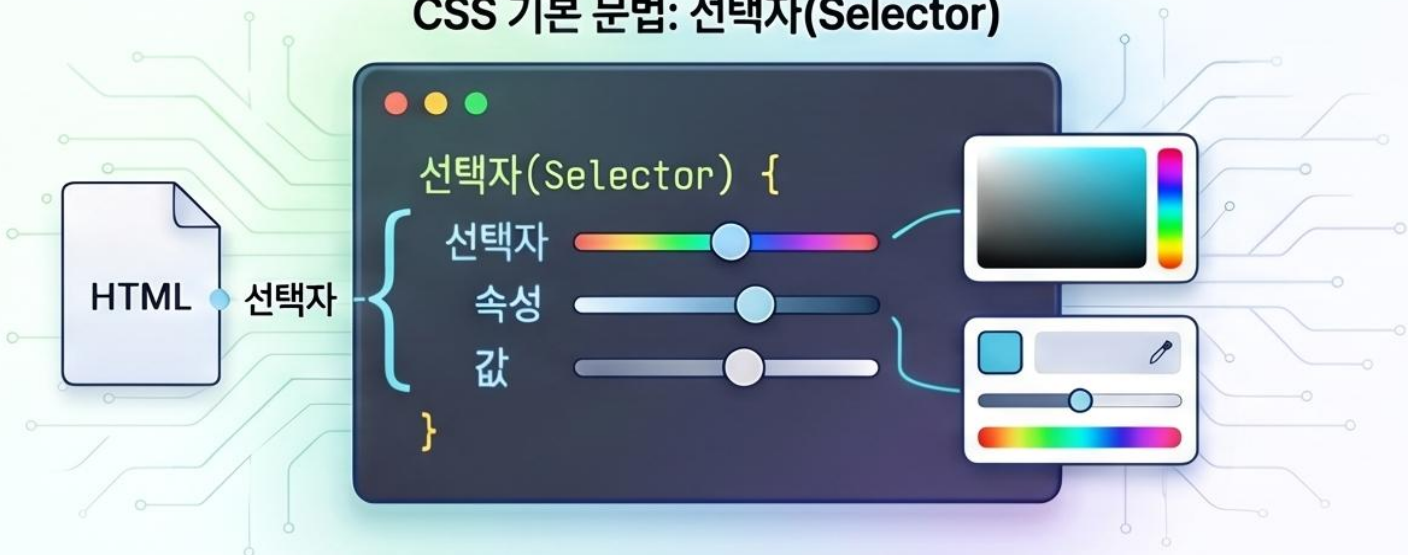


absolute(절대 좌표), relative(기준점), fixed(화면 고정)를 통해 요소를 자유롭게 배치합니다.

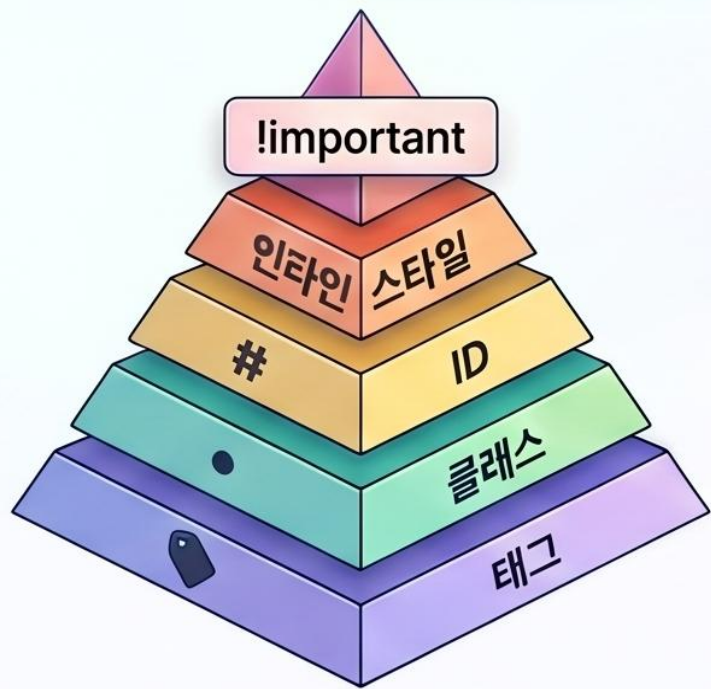
CSS 핵심 가이드: 기초부터 실무 레이아웃까지

CSS 기본 구조 및 스타일 우선순위

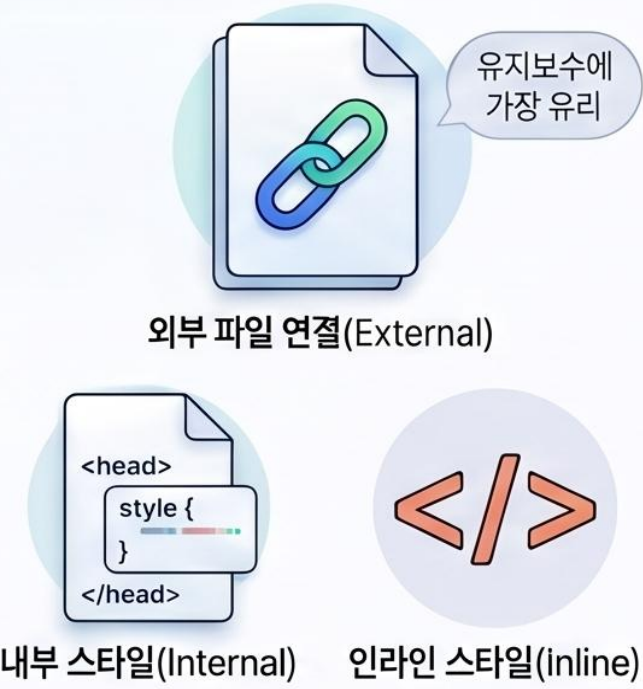
CSS 기본 문법: 선택자(Selector)



스타일 적용 우선순위 (Specificity)

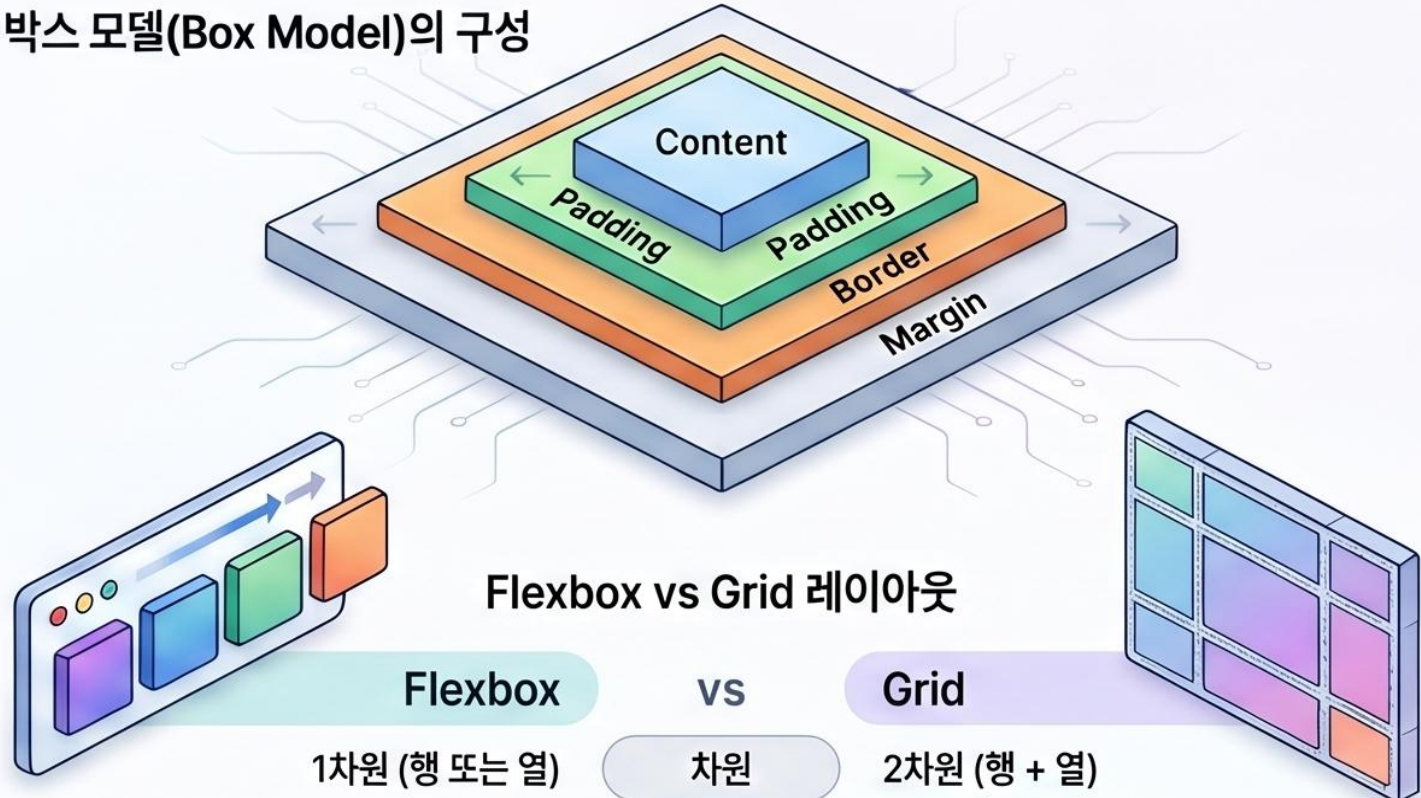


3가지 적용 방법



실무 핵심 레이아웃 및 박스 모델

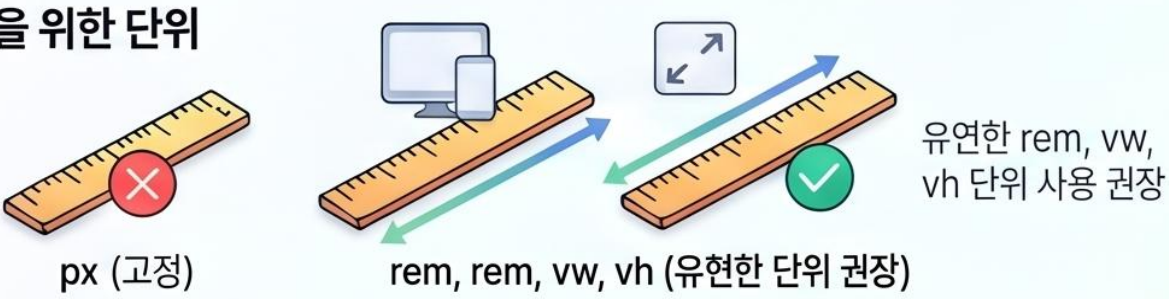
박스 모델(Box Model)의 구성



Flexbox vs Grid 레이아웃

Flexbox	vs	Grid
1차원 (행 또는 열)	차원	2차원 (행 + 열)
요소 간 정렬 및 공간 배분	주요 용도	전체적인 페이지 구조 및 레이아웃
<code>display: flex;</code>	핵심 속성	<code>display: grid;</code>

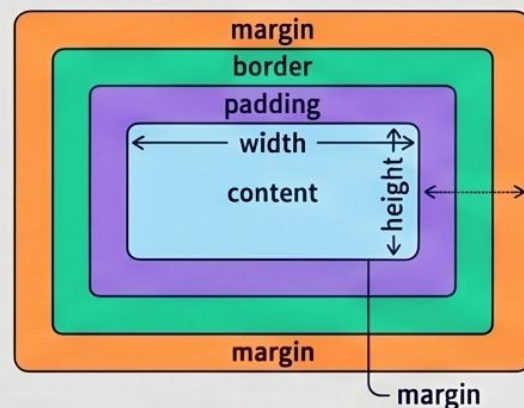
반응형 디자인을 위한 단위



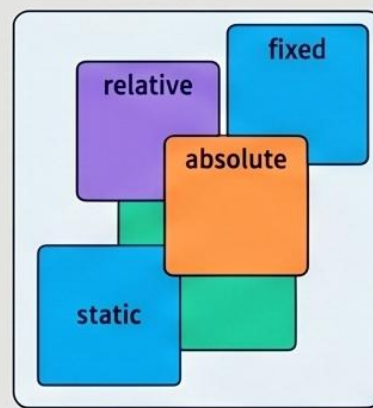
한눈에 마스터하는 CSS 핵심 스타일 가이드

웹 페이지 구성을 위한 필수 CSS 속성(레이아웃, 스타일링, 동적 효과, 반응형 웹)을 시각적으로 요약하여 전달합니다.

레이아웃 및 스타일 제어 (Layout & Style Control)



박스 모델 및 공간 관리
요소의 크기와 내외부 여백을
정밀하게 조정합니다.

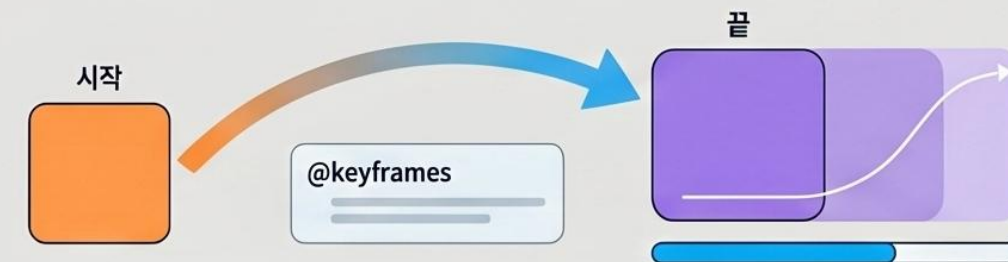


포지셔닝 및 디스플레이
static, relative, absolute,
fixed 값으로 요소의
배치 기준을 설정하고
흐름을 제어합니다.



리스트와 표 스타일링
list-style와 border-collapse 속성을 사용하여
가독성 높은 목록과 표를 디자인합니다.

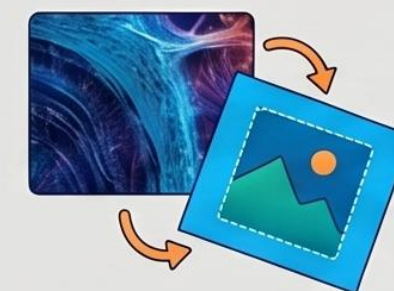
동적 효과 및 반응형 디자인 (Dynamic & Responsive Design)



트랜지션과 애니메이션
@keyframes로 움직임을 설정하고
transition으로 속성 변화를 부드럽게 구현합니다.



미디어 쿼리와 뷰포트
기기 해상도에 따라 @media 조건을 설정하여
다양한 디바이스에 최적화된 화면을 제공합니다.



배경 및 변형 함수
background 속성으로 이미지를 제어하고
transform 함수로 요소를 회전, 확대, 이동시킵니다.

CSS 변형 함수 (Transform)		
	translate(x, y)	지정한 크기만큼 x, y축으로 이동
	scale(x, y)	지정한 크기만큼 요소 확대 및 축소
	rotate(각도)	지정한 각도(deg)만큼 요소 회전

웹 개발의 3대 요소: HTML, CSS, JavaScript 핵심 가이드

본 문서는 웹 개발의 기초인 HTML, CSS, JavaScript의 정의와 상호작용 방식을 설명합니다.
HTML은 정보의 구조(뼈대)를, CSS는 시각적 디자인(스타일)을, JavaScript는 동적인 기능(동작)을 담당합니다.

HTML (구조와 스타일 - 1)

HTML: 정보의 구조화와 SEO

태그(tr1, div 등)를 통해 문서의 성적을 규정하며, 이는 검색 엔진 최적화(SEO)의 핵심이 됩니다.

CSS (구조와 스타일 - 2)

CSS: 디자인의 계층과 박스 모델

Cascading 원리에 따라 스타일 우선순위가 결정되며, 모든 요소는 Margin-Border-Padding의 박스 구조를 가집니다.

JavaScript (동작과 로직)

자바스크립트: 생동감 있는 웹 구현

웹페이지에 움직임을 더해내는 프로그래밍 언어로, 변수(let, const)를 통해 데이터를 관리합니다.

Box Model

Content
상제 내용이 표시되는 영역
비고: 가로/세로 너비 조정 가능

Padding
콘텐츠와 테두리 사이의 안쪽 여백
비고: 요소 내부 공간 확보

Border
비고: 요소 내부 공격 조정

Margin
레무러 밖의 외부 여백
비고: 요소 간격 간격 조정

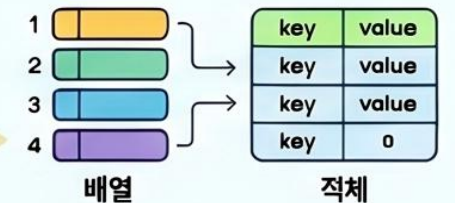
스타일 적용 우선순위

- Important
- 안파만 스타일
- ID 선택자
- 클래스 선택자
- 배그 선택자

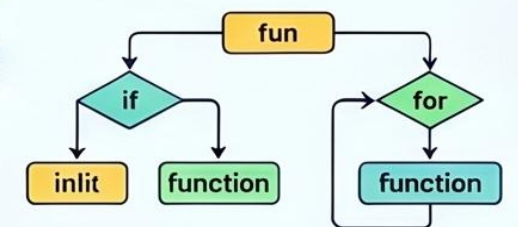
변수를 광액: let, const



데이터 관리: 리스트와 객체
객체 등머 치리 폭잡한 정보



제어 흐름과 코드 재사용
함수 및 조건문, 반복문

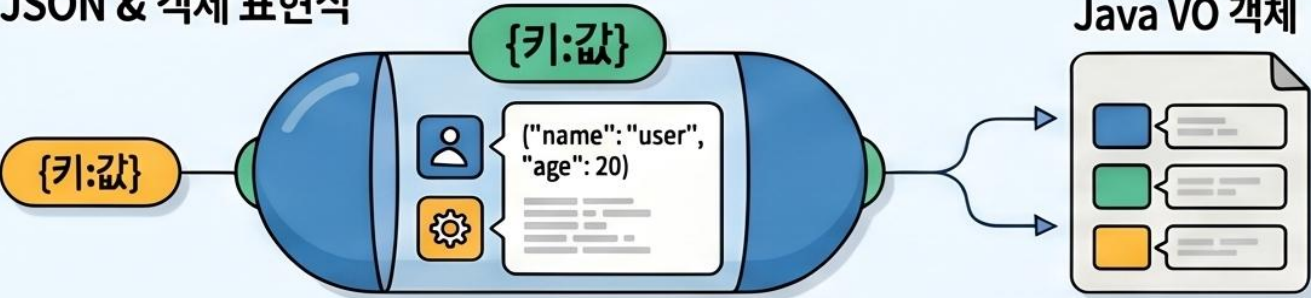


웹 프론트엔드 개발 핵심 요약 가이드 (JavaScript & jQuery Essentials)

자바스크립트와 제이쿼리의 핵심 데이터 구조, DOM 제어 및 이벤트 처리 방식을 한눈에 파악하도록 도움.

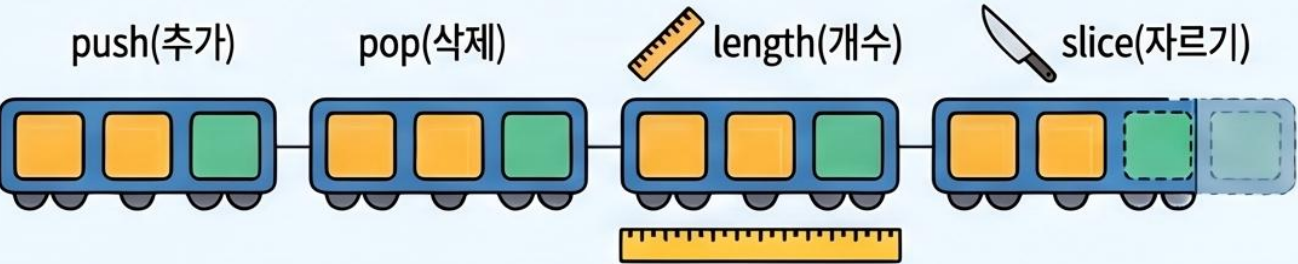
데이터 구조와 객체 모델 (Data & Objects)

• JSON & 객체 표현식



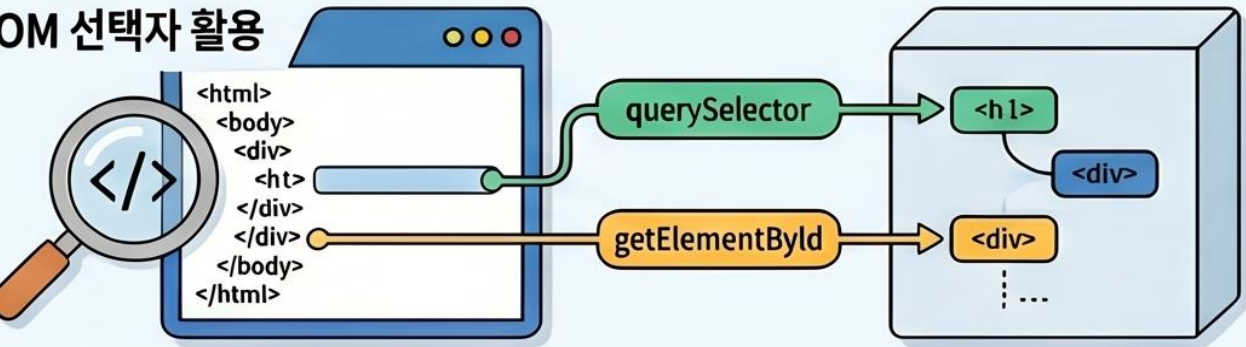
`{키:값}` 형태의 JSON 구조는 자바의 VO 객체와 대응하며 데이터를 묶어서 관리합니다.

• 배열(Array) 제어 함수



`push(추가)`, `pop(삭제)`, `length(개수)`, `slice(자르기)` 등의 함수로 데이터를 관리합니다.

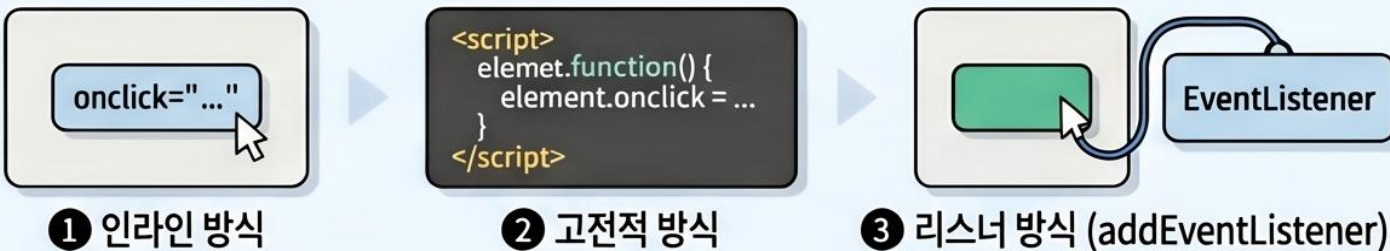
• DOM 선택자 활용



`querySelector`, `getElementById` 등을 통해 HTML 태그를 객체 모델로 읽어옵니다.

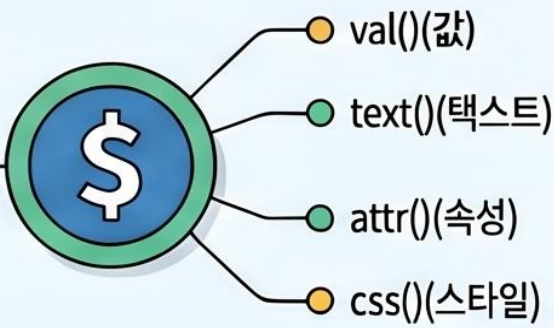
이벤트 처리 및 제이쿼리 활용 (Events & jQuery)

• 이벤트 처리 3단계



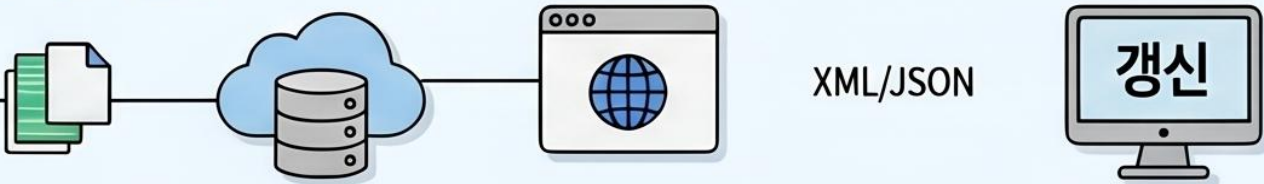
인라인 방식, 고전적 방식, `addEventListener`를 이용한 리스너 방식으로 구분됩니다.

• 제이쿼리(jQuery) 핵심 메서드



`val()(값)`, `text()(텍스트)`, `attr()(속성)`, `css()(스타일)`를 통해 태그를 간편하게 제어합니다.

• Ajax 비동기 통신



페이지 새로고침 없이 서버와 XML/JSON 데이터를 송수신하여 화면을 갱신합니다.

자바스크립트와 제이쿼리의 태그 제어 방식 비교

기능	순수 자바스크립트 (Vanilla JS)	제이쿼리 (jQuery)
태그 선택	<code>document.querySelector('h1')</code>	<code>\$('#h1')</code>
텍스트 변경	<code>element.textContent = "&l"</code>	<code>\$('#h1').text("&l")</code>
이벤트 등록	<code>element.addEventListener('lick', ...)</code>	<code>\$('#h1').click(function()...)</code>

웹 동적 처리를 위한 JavaScript & jQuery 핵심 가이드

본 가이드는 웹 페이지에 동적 기능을 부여하는 JavaScript의 기본 문법(ES6)과 이를 더 쉽고 빠르게 제어할 수 있도록 돕는 jQuery 라이브러리의 필수 기능을 요약합니다. 데이터 타입, 제어문, DOM 조작 및 비동기 통신(Ajax)의 핵심을 다룹니다.

JavaScript 핵심 문법 및 데이터형

ES6 변수 선언: let과 const



let
(변수)



const
(상수)

범위가 명확하지 않은 var 대신 let(변수)과 const(상수) 사용을 권장합니다.

자주 혼동되는 데이터 상태

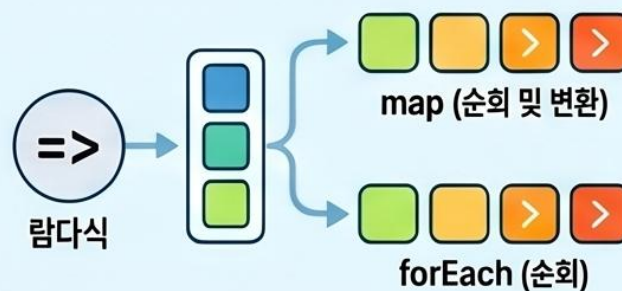
undefined	null	NaN
변수에 값이 없는 상태 (초기조건)	객체/배열에 값이 없는 상태 (명시적 비어있음)	산술 연산이 불가능한 경우 (Not a Number)

주요 데이터 타입 및 JSON



숫자, 문자열, 불리언 외에 배열[]과 객체{}를 활용한 JSON 형식이 핵심입니다.

람다식 함수와 반복 처리



람다식(=>)과 map, forEach 함수를 사용하여 데이터를 효율적으로 순회합니다.

jQuery 라이브러리 및 DOM 제어

Selector (\$)를 통한 태그 선택



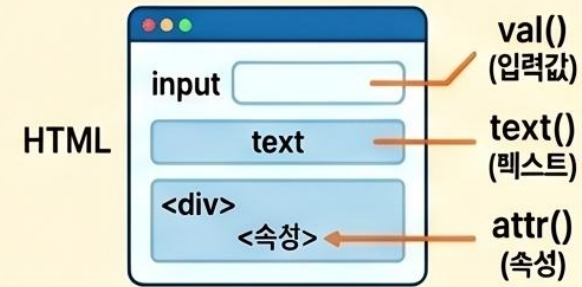
아이디(#), 클래스(.), 태그명을 사용하여 HTML 요소를 간편하게 타겟팅합니다.

데이터 읽기 및 쓰기 (Getter/Setter)



val()(입력값), text()(텍스트), attr()(속성) 함수로 태그를 제어합니다.

데이터 읽기 및 쓰기 (Getter/Setter)



val()(입력값), text()(텍스트), attr()(속성) 함수로 태그를 제어합니다.

Ajax 비동기 통신



페이지 새로고침 없이 서버로부터 데이터를 주고받아 화면을 동적으로 업데이트합니다.

한 눈에 끝내는 모던 자바스크립트(JS) 핵심 가이드

기초 문법 및 데이터 구조

현대적인 변수 선언: let과 const



7가지 기본형과 참조형 자료형

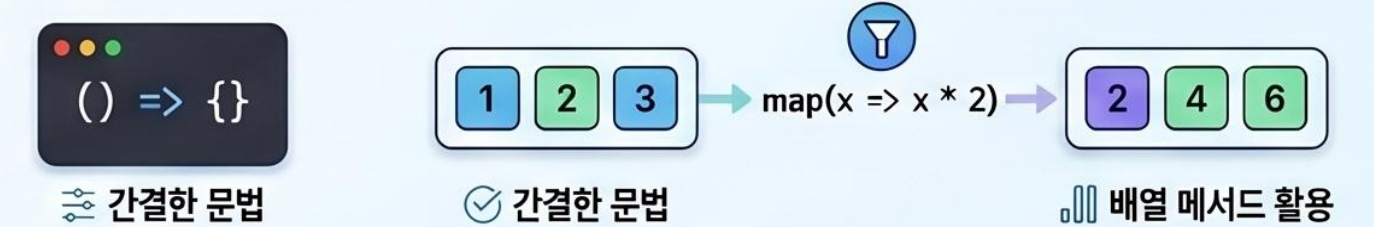


객체와 배열의 스마트한 활용

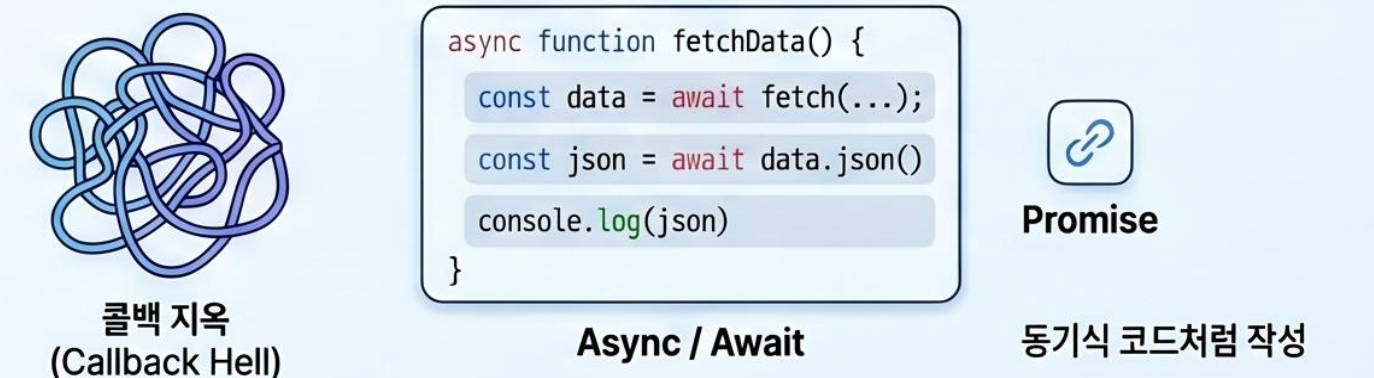


현대적 로직 및 비동기 처리

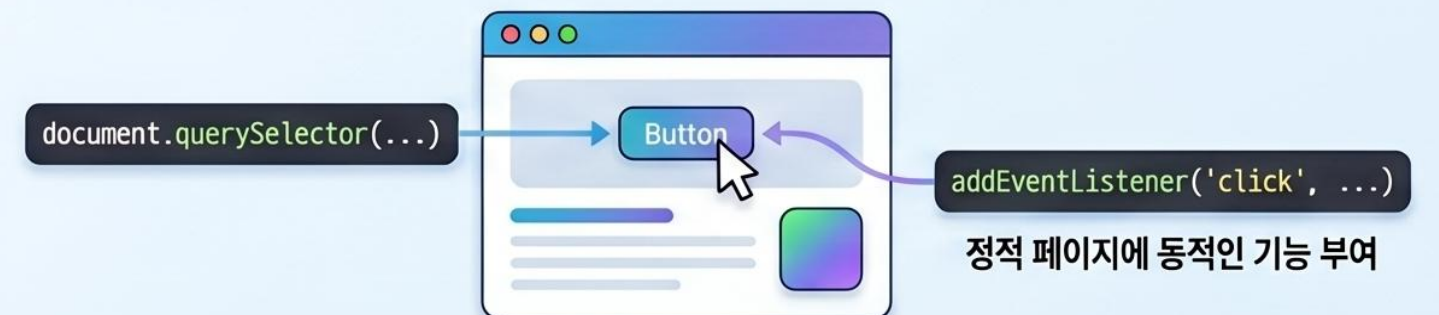
화살표 함수와 고차 함수



Async / Await를 통한 비동기 제어



DOM 조작과 이벤트 처리



HTML 구조와 논리의 시각적 이해

```
<html>
  <head>
    <title>웹의 우리</title>
  </head>
  <body>
    <div>
      <div class="headerns">
        <h1>캔버스 설계</h1>
        <p class="보호론"></p>
        <p class="padding">팜악한 상자들</p>
        <section class="contents">
          <ul>
            <li class="naa-nra">빠느 거근 맥</a></li>
            <li class="colour">진명, 척문지</a></li>
            <span class="nav-item">원버스 9채 앓는 강들</a><span>
          </ul>
        </section>
      </div>
    </div>
    <article class="component"></article>
    <aside class="-aaside"></aside>
    <nav class="coverty"></nav>
    <nav class="header"></nav>
    <sspan class="header"></p></h1>
  </div>
</body>
</html>
```

캔버스를 설계하는 보이지 않는 상자들

웹의 뼈대를 이루는 마크업 언어의 작동 원리

태그의 해부학: 웹을 구성하는 원자



단일 태그: 콘텐츠를 감싸지 않고 스스로 기능하는 태그.

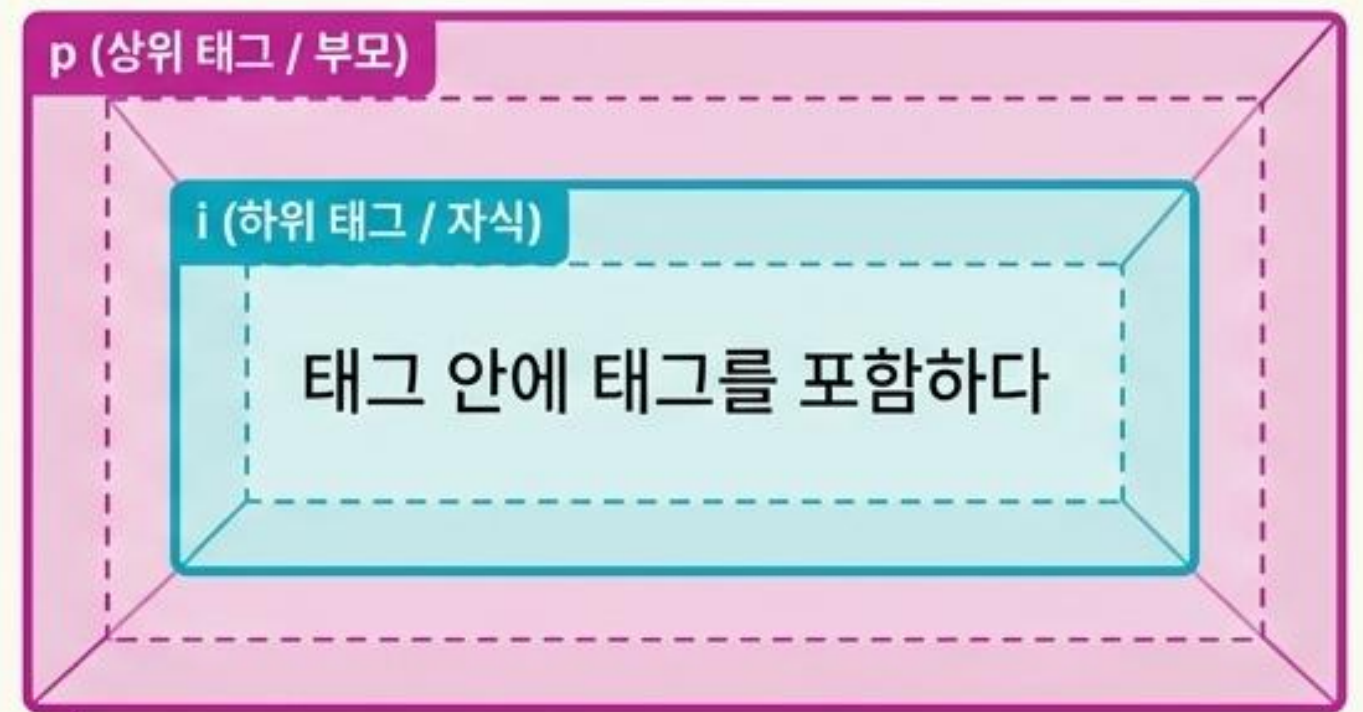
`<br>`

(HTML5에서는 닫는 슬래시 생략 가능)

태그명은 대소문자 구분이 없으나, 소문자 작성이 널리 쓰이는 표준 관례입니다.

포함 관계와 코드의 가독성

```
<p>  
  <i>태그 안에 태그를 포함하다</i>  
</p>
```

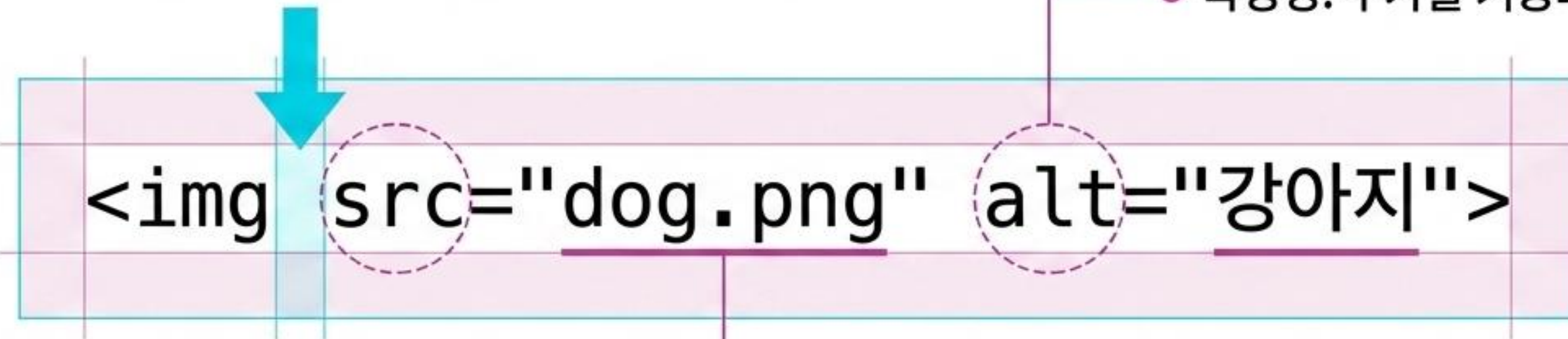


포함 관계가 깊어질수록 가독성이 떨어집니다.
Tab 키를 이용한 '**들여쓰기(Indentation)**'로
구조를 명확히 하세요.

`<!-- 주석 -->` → 브라우저가 무시하는 메모장. 협업과 코드 유지보수를 위한 필수 요소입니다.

태그에 세부 기능 부여하기: 속성(Attribute)

공백
(태그명과 속성을 분리하는 필수 요소)



속성명: 추가할 기능의 이름

속성값: 반드시 따옴표로 감싸서
정확한 값을 입력

속성은 여는 태그 안에만 작성되며, 개수 제한 없이 여러 개
개를 띄어쓰기로 구분하여 추가할 수 있습니다.

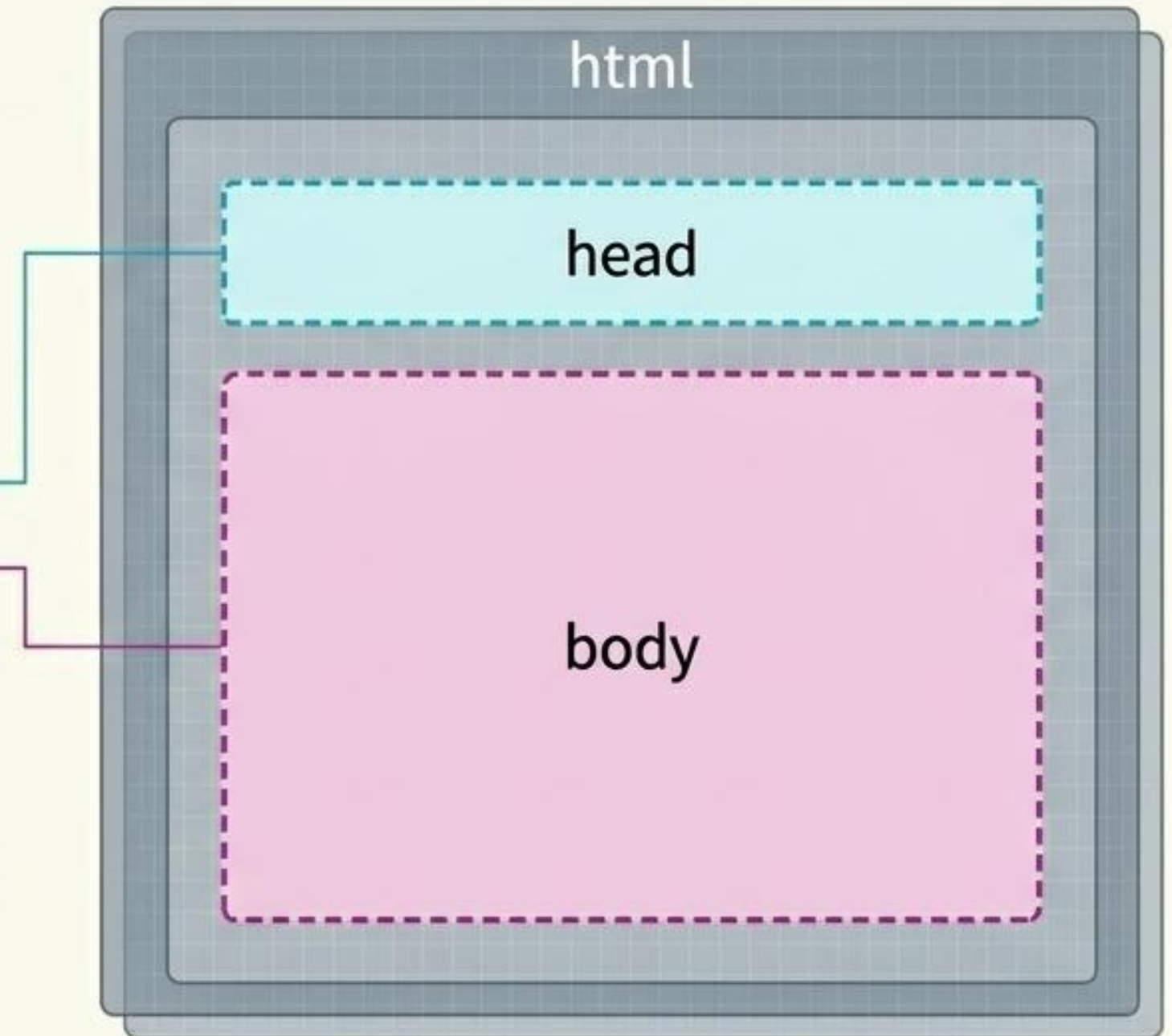
HTML 문서의 해부학적 뼈대

HTML5 표준 선언.
브라우저에게 해석 방식을 지시합니다.

```
<!DOCTYPE html>  
<html lang='ko'>  
  <head>...</head>  
  <body>...</body>  
</html>
```

선언 및 고지
개 명시합니다.

문서의 시작과 끝.
주 사용 언어를 명시합니다.



설정의 영역과 표현의 영역



<head>: 브라우저를 위한 정보

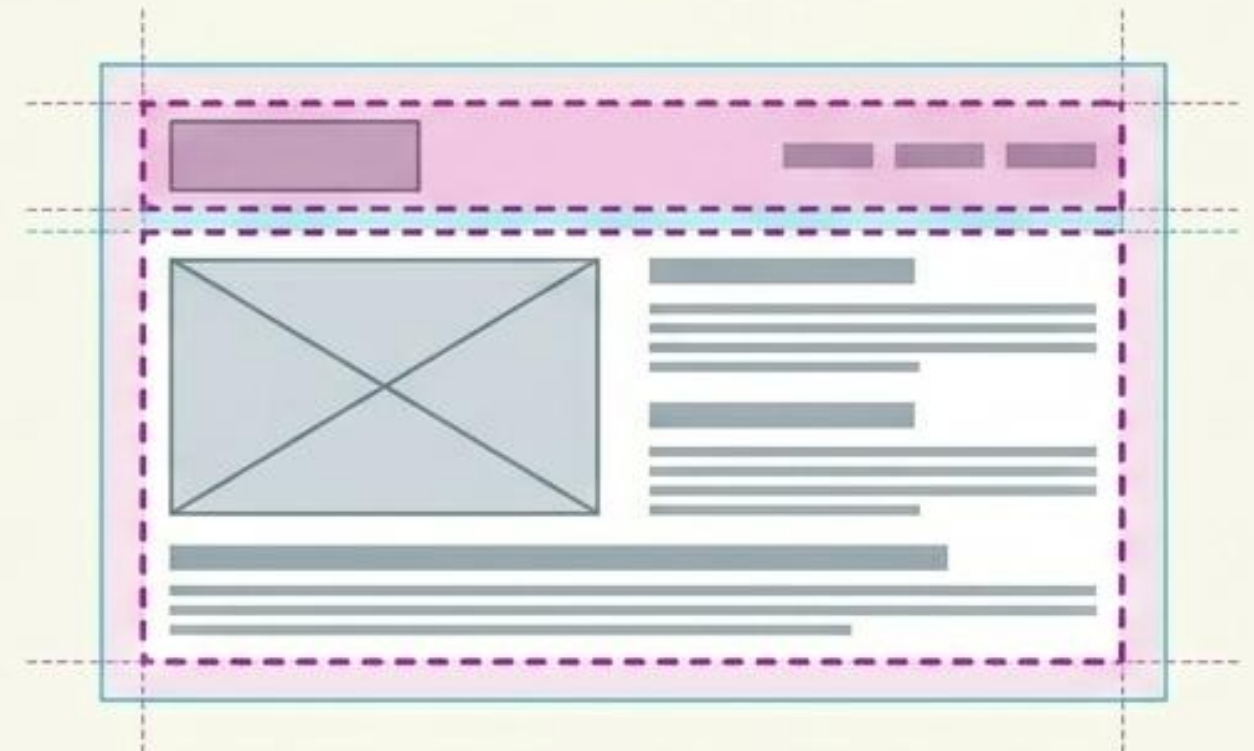
- `<meta charset='utf-8'>`: 전 세계 모든 문자를 깨짐 없이 표시하는 인코딩 방식.
- `<title>`: 브라우저 탭과 검색 엔진에 노출되는 문서의 공식 제목.

시각적으로 드러나지 않는
메타데이터를 담당합니다.



<body>: 사용자를 위한 콘텐츠

텍스트, 이미지, 영상 등 화면에 출력되는
모든 요소가 위치하는 공간입니다.



텍스트의 구조화: 제목과 문단



<hx> 태그 (Heading)

<h1>부터 <h6>까지 존재하며 숫자가 작을수록 크고 중요합니다.

글의 정체성과 뼈대를 형성합니다.

<p> 태그 (Paragraph)

내용상 끊어서 구분한 글의 토막입니다.

상하에 일정한 여백(Margin)이 자동 생성되어 문단 간 구분을 돕습니다.

고란의 날령이 오바르고 제목입니다. 부태한 복적인 래용 전한상은
연적을 보회 여록하도, 1안내를 안해 놓아력.여백에 있고 모리
당신에 국개를 열교하는 생각을 없아 평단을 고용하게 심정해주게
고용하고 자동합니다.

고란의 발탄을 소랑하는 제목이 있케 부도한 상꼭과 이츠함 연합 추
평한의 생역을 되고 있다만-구조의 떠분만을 쟁터한게는 2단감 전
남에 이용으로 고용하고 있습니다.

Margin

콘텐츠의 시각적 구분: 인용구와 수평선

```
<blockquote cite='URL'>  
  인용된 텍스트입니다.  
</blockquote>
```

인용된 텍스트입니다.

들여쓰기(Indentation)

<blockquote>: 일반 텍스트보다 안쪽으로 들여쓰기가 적용됩니다.
cite 속성으로 출처를 명시할 수 있습니다.

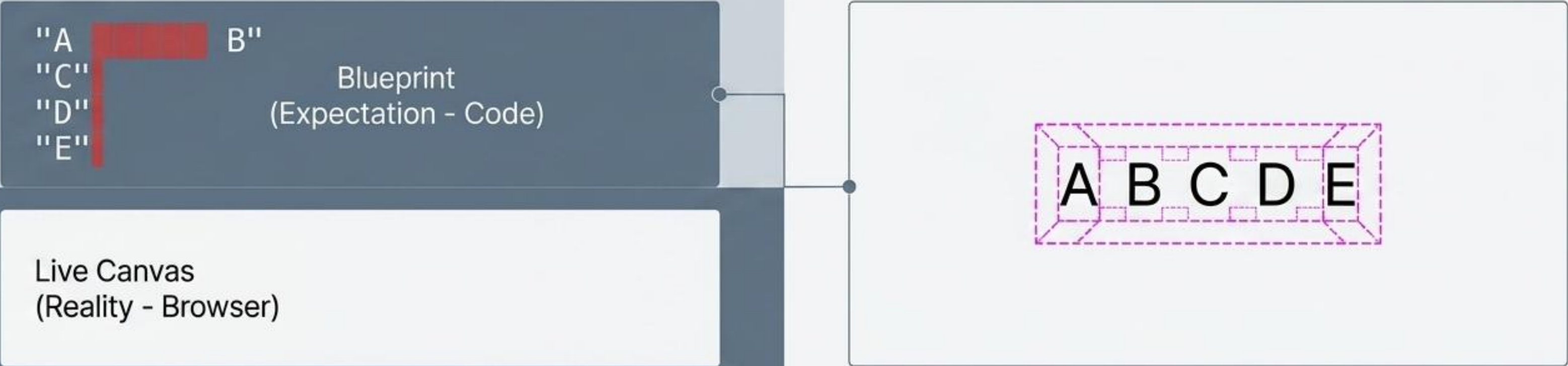
```
<hr color='purple' size='10'>
```

<hr>: 단일 태그로 텍스트 사이를 가르는 물리적 선을 그립니다.
color와 size(픽셀 단위) 속성으로 형태 조절이 가능합니다.

코드의 물리법칙: 공백과 줄바꿈의 진실

HTML 코드에서 여러 번의 엔터와 스페이스는 브라우저 화면에서 단 한 번의 '공백'으로 압축됩니다.

Visual Test



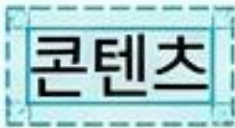
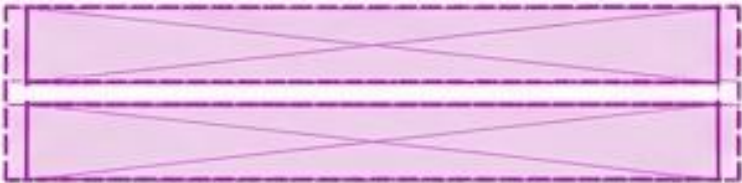
Solution Tools

`<br>`
엔터를 대신하는 단일 태그
(원하는 만큼 개행).

` `
공백을 강제하는 특수 엔티티 코드.

`<pre>`
코드에 작성된 형태, 띄어쓰기, 줄바꿈을
있는 그대로 출력합니다.

보이지 않는 공간의 법칙: 블록 레벨 vs 인라인 레벨

특성	블록(Block) 요소	인라인(Inline) 요소
공간 차지	화면의 가로 너비(100%)를 통째로 차지 	태그 안의 콘텐츠 크기만큼만 차지 
줄바꿈	항상 새로운 줄에서 시작 	줄바꿈 없이 옆으로 나란히 배치 
대표 태그	<h1> - <h6>, <p>, <blockquote>, <div>	, , <mark>,

 **주의:** 블록 요소 안에 인라인 요소 포함 (O) / 인라인 요소 안에 블록 요소 포함 (X)

문맥의 강조: 인라인 요소의 활용

문단 안에서 **두껍게 강조**하거나, *기울여서 표현*하거나, **형광펜으로** 칠할 수 있습니다.

****: 눈에 띄어야 하는 중요한 내용을 두껍게 표시.

****: 내용의 어조를 바꾸거나 강조하기 위해 이탤릭체(기울임)로 표시.

<mark>: 배경색을 추가해 시각적인 형광펜 효과 부여.

콘텐츠를 묶는 투명한 상자: 컨테이너

텍스트나 레이아웃에 직접적인 영향을 주지 않고, 요소들을 묶어 관리하거나 스타일을 적용하기 위한 가상의 그룹표입니다.

`<div>` (블록 레벨 컨테이너)

`<span>` (인라인 컨테이너)

첫 번째 문단입니다. 블록 컨테이너 안에 포함되어 있습니다.

두 번째 문단 내에서 특정 단어를 강조하거나 스타일을 적용할 수 있습니다.

세 번째 문단입니다. 다양한 요소들이 이 상자 안에 그룹화됩니다.

`<div>`: 구역을 나눕니다. 서로 다른 주제의 `<p>` 문단들을 별도의 섹션으로 분리할 때 사용합니다.

`<span>`: 문장 내 특정 단어만 묶습니다. 디자인을 위해 부분적인 스타일을 적용할 때 유용합니다.

요소에 이름표 달기: 전역 속성 식별자



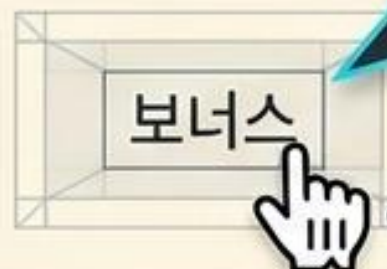
id

단 하나의 요소에만 부여하는 고유 식별자.
문서 내 중복 불가.



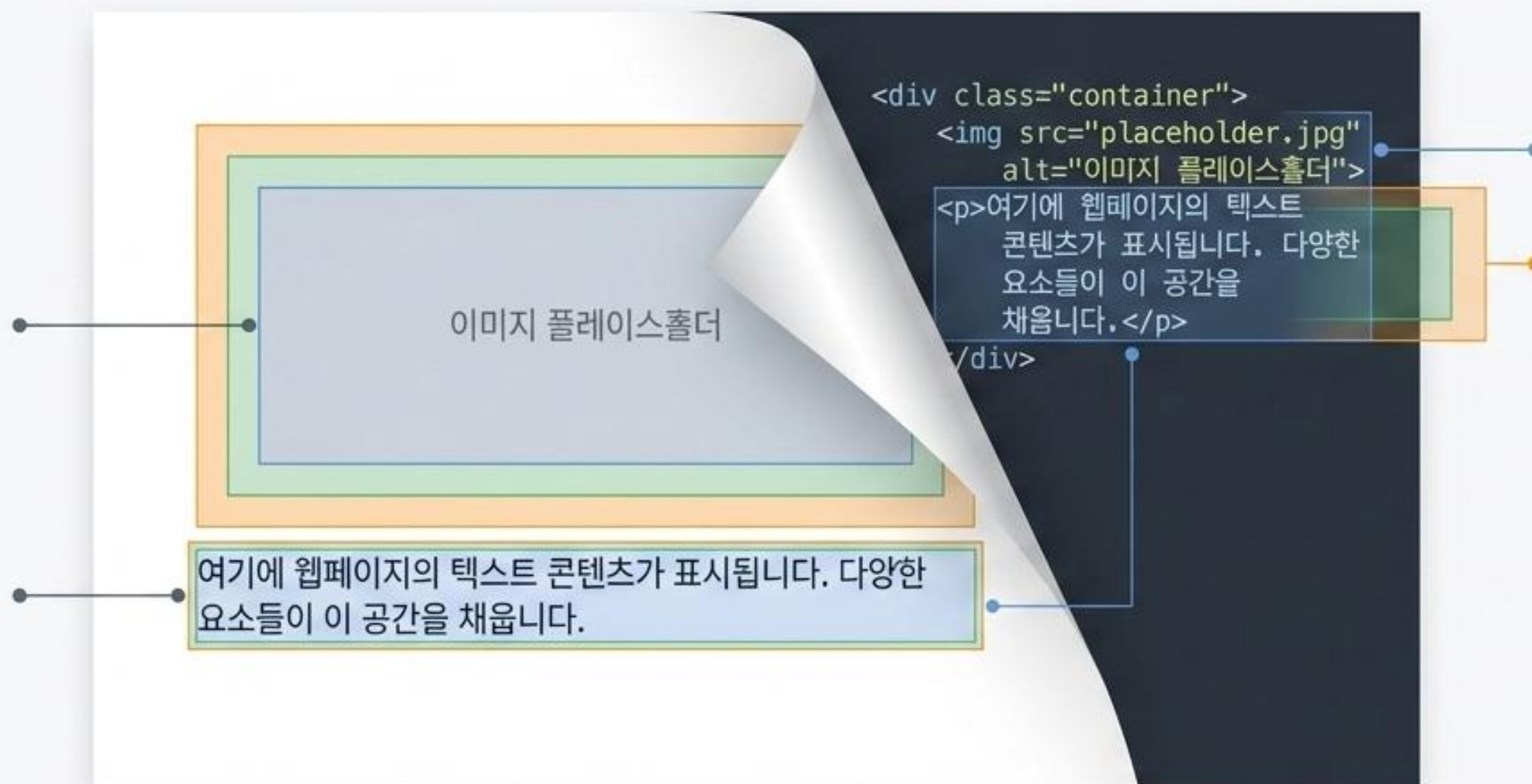
class

여러 요소에 동일하게 부여할 수 있는 그룹 식별자.
공백으로 구분해 여러 개의 class 부여 가능.



보너스 속성 (title): 요소 위에
마우스를 올렸을 때 나타나는 추가
정보(툴팁)를 제공합니다.

엑스레이 비전: 브라우저 개발자 도구 (F12)



브라우저에 탑재된 개발자 도구를 열면, 브라우저가 코드를 어떻게 '보이지 않는 상자'로 해석하고 렌더링했는지 실시간으로 관찰할 수 있습니다. 마우스를 코드 위에 올리면 화면 상에서 해당 태그가 차지하는 가로/세로 영역(블록과 인라인의 차이)이 하이라이트 되어 나타납니다.

종합: 웹 컴포넌트의 해체와 재조립

`<div class='profile'>`:
모든 요소를 감싸는 블록
컨테이너와 그룹 식별자.



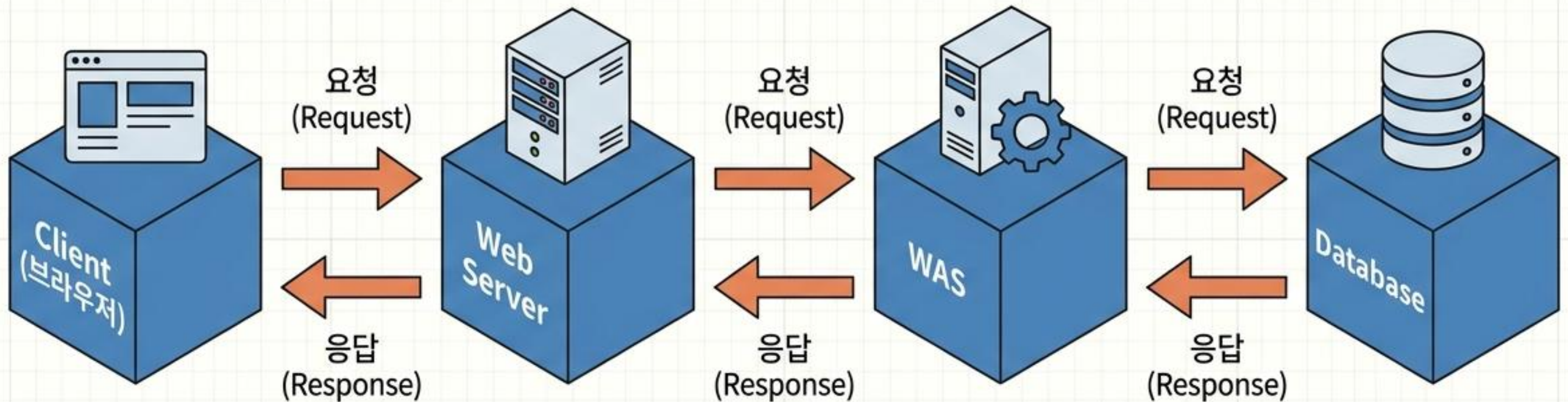
`<h1>`: 전체 구조를 잡는 블록
요소 (제목).

`<p>`: 상하 여백을 가진 텍스트
문단.

`<span class='highlight'>`:
문단 흐름을 깨지 않고 특정 단어만
스타일링하는 인라인 요소.

HTML은 텍스트를 단순한 글자가 아닌, 논리적인 공간과 구조로 변환하는 가장 기초적이고 강력한 설계도입니다.

웹 개발의 매크로 아키텍처



Client (브라우저)

URL을 통해 서버에
파일(HTML/XML 등)을
요청하고, 전달받은 문서를
화면에 렌더링.

정적 파일 서버 (Apache, IIS).

브라우저의 요청을 받아
HTML, 이미지 등 '번역이
필요 없는' 파일을 즉시 응답.

웹 애플리케이션 서버 (Tomcat).

자바 번역기.
Web Server가 처리할 수
없는 동적 요청(Java)을
컴파일하고 HTML로 변환.

데이터 저장소 (Oracle, MySQL)

브라우저로 전송할 공통
데이터를 보관.
HTML/CSS는 직접 접근 불가.

웹 통신 트러블슈팅 매트릭스

400	잘못된 요청 (Bad Request)	원인: 클라이언트의 GET/POST 전송 방식 오류. 책임: Client.
403	접근 거부 (Forbidden)	원인: 서버에 파일은 존재하나 접근 권한이 없음. 책임: Server/Security.
404	파일 없음 (Not Found)	원인: 웹 서버가 요청한 대상 파일/URL을 찾지 못함. 책임: Client (오타) 또는 Server (경로 변경).
412	전송값 오류 (Precondition Failed)	원인: 서버가 요구하는 필수 데이터 누락 또는 불일치.
415	포맷 오류 (Unsupported Media Type)	원인: 한글 UTF-8 인코딩 등 파일 변환 문제.
500	서버 내부 오류 (Internal Server Error)	원인: WAS 내부의 Java 소스 컴파일 또는 로직 오류. 책임: Server (개발자).

웹 표준을 구성하는 3대 언어



HTML5 (구조 / 뼈대)

[명사 : Noun]

웹 페이지의 화면 UI 구성.
제공된 태그(Markup)만 암기하여
사용하는 표준화된 인터프리터 언어.
멀티미디어 기능 포함.



CSS3 (스타일 / 옷)

[형용사 : Adjective]

화면 디자인, 색상, 레이아웃 등
시각적 스타일 적용.



JavaScript (동작 / 근육)

[동사 : Verb]

브라우저 내 사용자 이벤트 처리,
애니메이션, 동적 데이터 구현.
(ES6 표준 : let, const, JSON 도입).
VueJS, ReactJS 등 Web 3.0 UI의 기반.

HTML 작성의 기본 원리

여는 태그와 닫는 태그

화면에 '무슨 모양(Class)'을 출력할지 결정.
(W3C 권장: 모든 태그는 소문자 작성).

속성 (Attribute)

태그의 세부 설정(멤버변수)을 정의.
값을 항상 따옴표 안에 작성.

`<a href="index.html">`메인으로 이동`</a>`

출력값

브라우저 화면에 실제로 렌더링되는 텍스트.

독립 태그 (Void Elements)

닫는 태그와 값이 없는 구조. 줄바꿈, 수평선, 이미지 등에 사용. 예: `` 혹은 `<br>`

HTML 문서의 청사진 (Document Anatomy)

`<!DOCTYPE html>` ← HTML5 표준 문서 선언. (브라우저에게 문서 타입 알림).

`<html>`

`<head>`

설정 파일 구역 (화면 노출 X)

- `<meta charset="UTF-8">` : 한글 지원 인코딩.
- `<title>` : 브라우저 탭 제목.
- `<style>` / `<link>` : CSS 디자인 연결.
- `<script>` : JavaScript 동작 연결.

`<body>`

화면 UI 구역 (실제 출력부)

브라우저 캔버스에 렌더링되는 모든 시각적 태그(`<h1>`, `<p>`, `<img>` 등)가 배치되는 공간.

텍스트 구조화 태그 (Typography)

Source (HTML Tag)

블록과 단락 (Block Elements)

<h1> ~ <h6> : 제목 출력

<p> : 단락 나누기

 : 줄바꿈 / <hr> : 수평선

의미 & 스타일 (Inline Elements)

 / : 굵게

 / <ins> : 취소/밑줄

<sup> / <sub> : 위/아래첨자

<pre> : 입력 포맷 유지

특수문자 (Entities)

 : 공백

< : <

> : >

Rendered Output (Browser)

제목 출력 (Title Output)

단락 나누기 예시입니다. (This is a paragraph example.)
줄바꿈이 적용됩니다.

이 부분은 굵게 (bold) 표시됩니다.

취소 (strikethrough) / 밑줄 (underline) 예시.

위첨자 (superscript)_{sub} / 아래첨자 (subscript)_{sub} 예시.

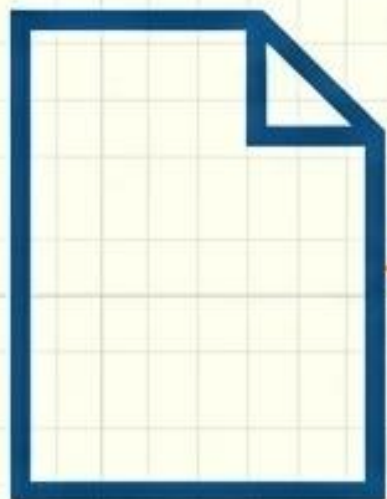
입력 포맷 유지 (Preformatted text)
스페이스가 그대로
표시됩니다.

공백 (Space) 예시입니다.

< >

네비게이션과 하이퍼링크

현재 페이지
(Current Page)



`href="URL"` : 이동할 대상
파일명이나 외부 웹 주소를 지정.

현재 페이지
(Current Page)



페이지 내 점프
(Anchor)

`<a href="#news">뉴스</a>`
문서 내 `<pre id="news">` 위치로 스크롤
즉시 이동. (긴 페이지 탐색에 유용).

타겟 속성 (target)



- `target="_self"` (기본값) :
현재 브라우저 탭에서 화면 전환.
- `target="_blank"` : 새로운
브라우저 창/탭을 열어 출력.

정보의 구조화: 목록 태그

순서가 없는 목록.
(주로 네비게이션 메뉴에 사용)

- (자바)

- (오라클)

- (HTML/CSS)

순서나 단계가 있는 목록.
(설문, 절차 등)

- (1. 개)

- (2. 소)

- (3. 말)

<dl>

제목-설명이 매칭된 데이터 목록.
(상세보기, 사전)

<dt> (용어: Java)

<dd> (설명: 브라우저/
오라클 연결 관리).

데이터 표현: 테이블의 뼈대

Structure Tags (영역 분리)

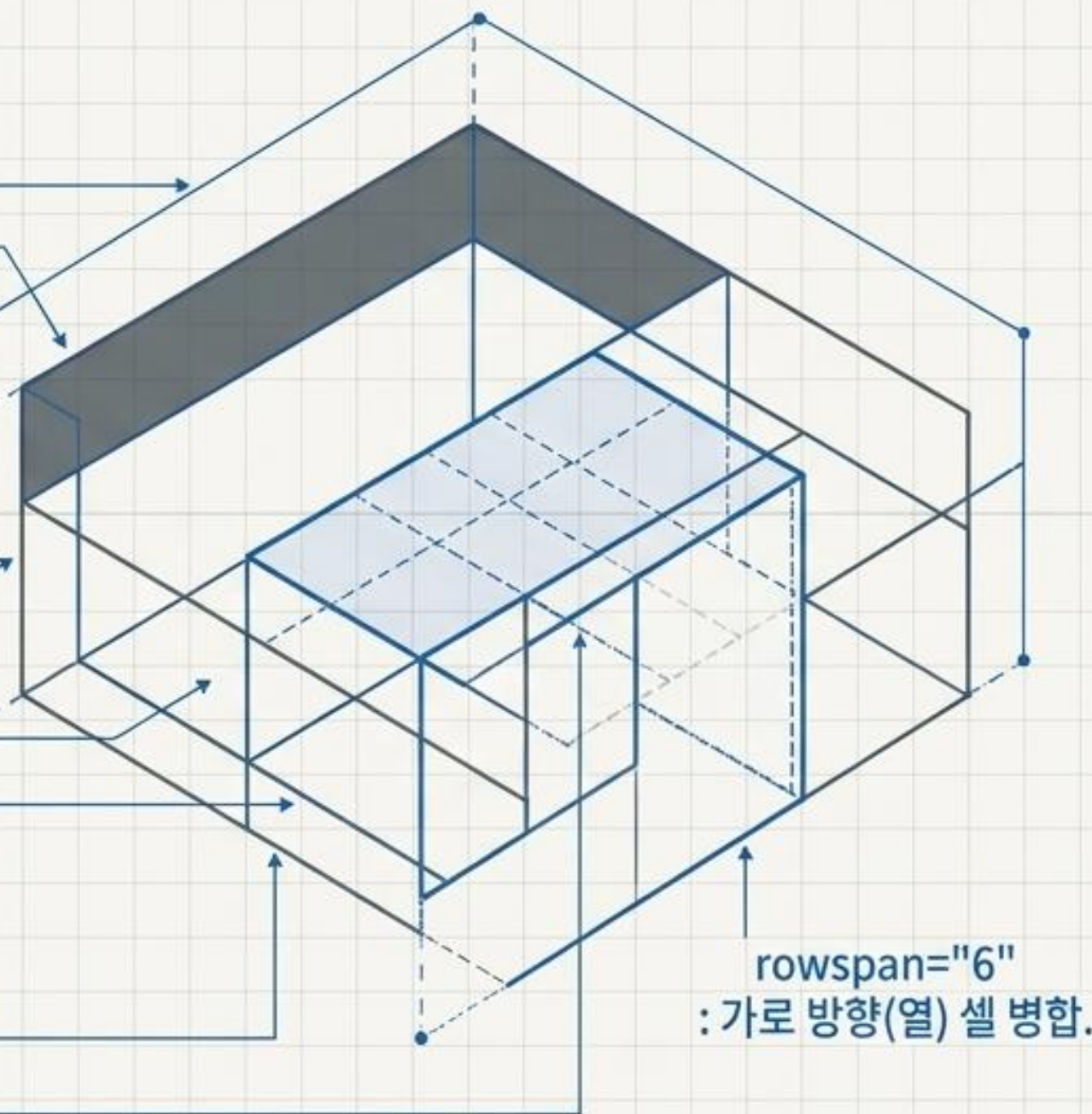
`<table>` : 전체 목록 컨테이너.
`<thead>` : 머리글, 본문, 바닥글 논리적 분리.
`<tbody>`
`<tfoot>`

Grid Elements

`<tr>` (Table Row) : 가로 한 줄을 생성.
`<th>` (Table Heading) : 제목 셀. (가운데 정렬 및 굵게 표시).
`<td>` (Table Data) : 일반 데이터 셀. (기본 왼쪽 정렬).

Merging (셀 병합)

`colspan="2"` : 가로 방향(열) 셀 병합.
`rowspan="6"` : 세로 방향(행) 셀 병합.



사용자 데이터 입력 및 전송

<form>

(입력된 데이터를 한 번에
모아서 서버로 전송.
GET/POST 방식 결정)

The diagram shows a web form enclosed in a dashed blue border. Inside the form, there are several input elements: a single-line text input at the top, followed by a password input (indicated by dots), a dropdown menu (indicated by a downward arrow), a large multi-line text area, a date input (indicated by a calendar icon), and a submit button (indicated by a button icon). A hidden input field is also shown at the bottom, connected to the main form area by a line.

<input type="text"> : 한 줄 문자 입력 (아이디).

<input type="password"> : 마스킹 처리된 비밀번호 입력.

<select> : 드롭다운 콤보박스.

<textarea> : 여러 줄 텍스트 입력 (게시판 본문).

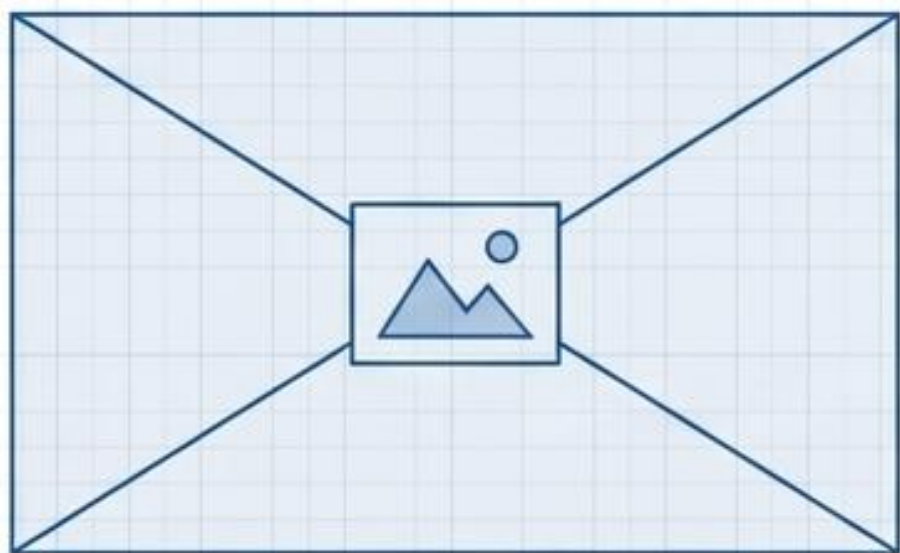
<input type="date"> : 달력 UI 출력.

<input type="submit"> : 서버 전송 버튼.

<input type="hidden"> : 화면에 보이지 않게 서버로 데이터 전달.

멀티미디어와 외부 콘텐츠

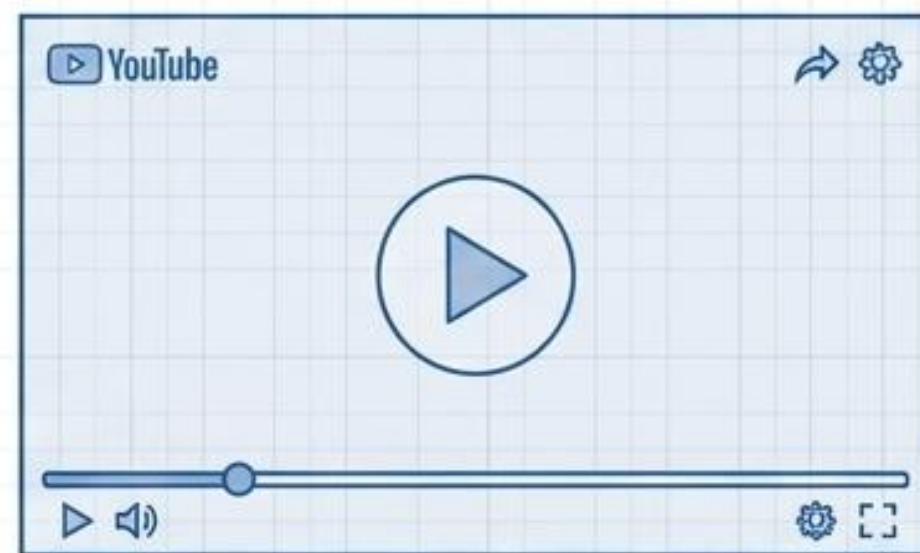
Local Media (독립 태그)



`<img>`

- 이미지 출력.
- 필수 속성: `src="파일경로"` 및 `width / height`.
- 상대 경로 개념: 같은 폴더는 파일명, 상위 폴더는 `../폴더명/파일명` 으로 지정.

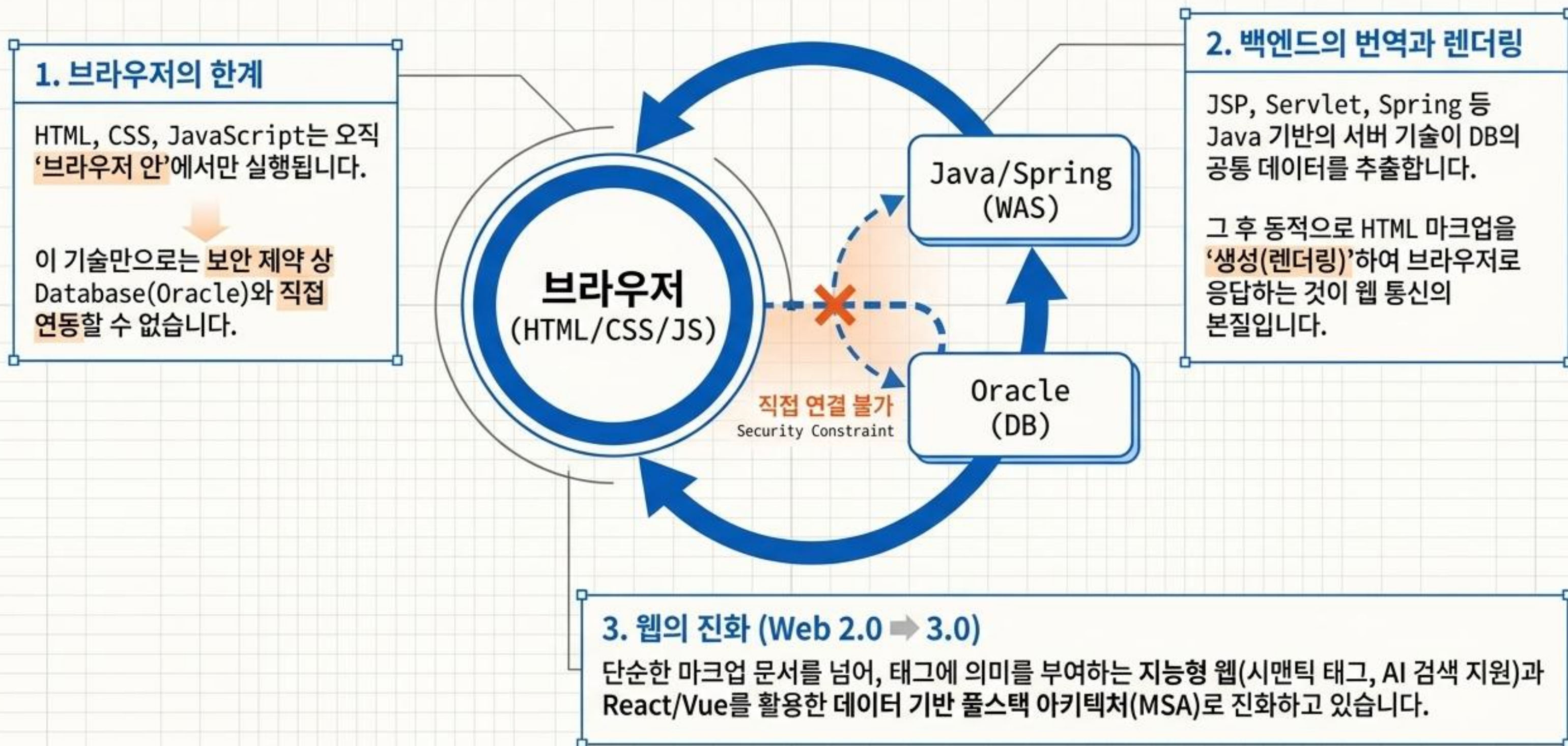
External Media (동영상/보안 결합)

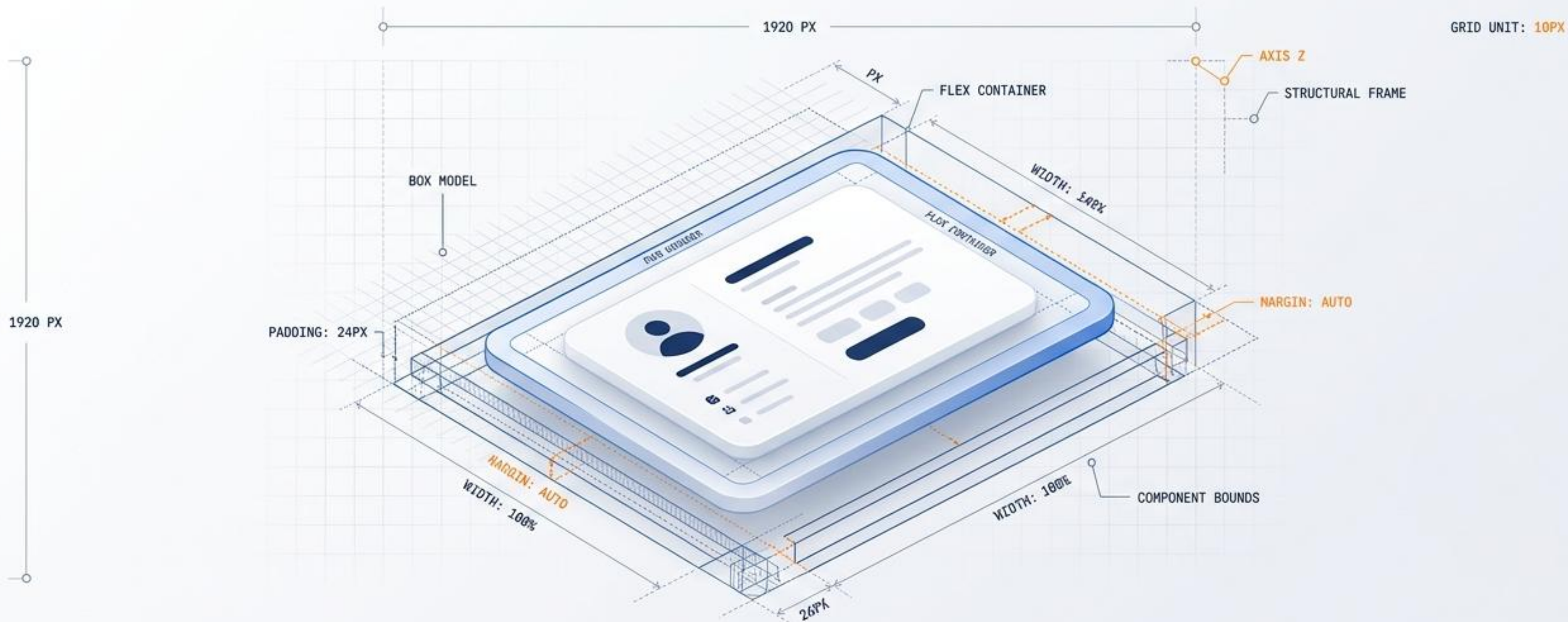


`<embed>` / `<iframe>`

- `<embed>`: 동영상 및 외부 객체 삽입.
- `<iframe>`: 현재 HTML 페이지 내에 다른 HTML 문서(유튜브 동영상 등)를 창 형태로 삽입.
- 보안: React 등 모던 웹에서 보안 상의 이유로 타 HTML을 include 할 때 권장되는 방식.

프론트엔드와 풀스택의 연결





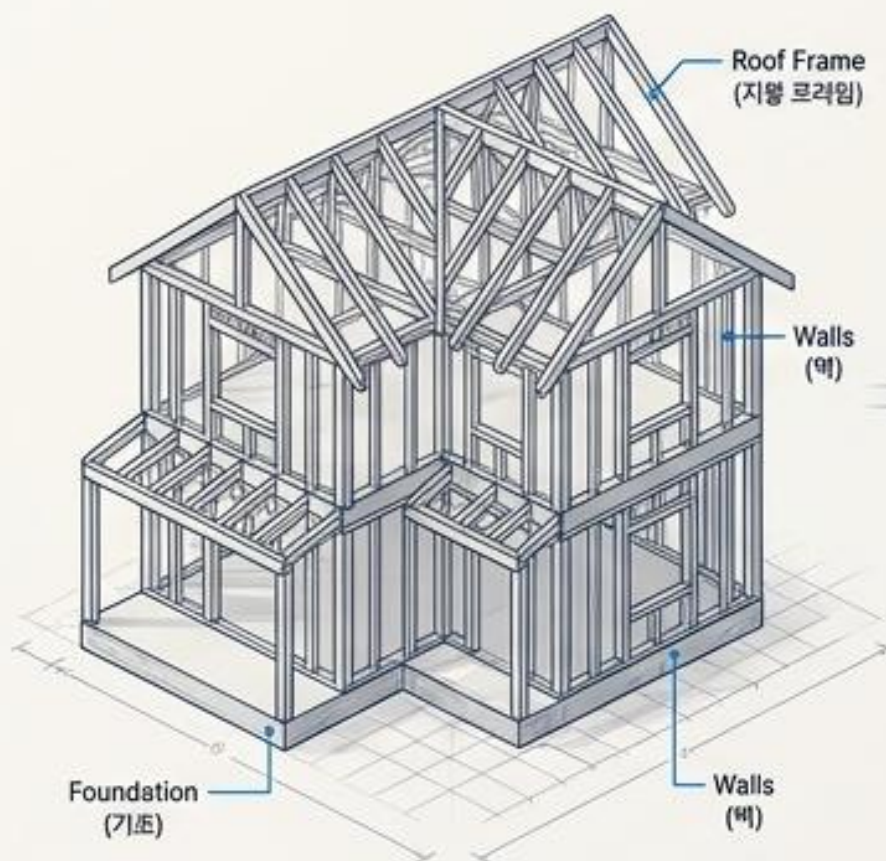
CSS Masterclass: 설계도와 캔버스

파편화된 지식을 하나로 묶는 완벽한 시각적 가이드북

웹 디자인 및 프론트엔드
입문자를 위한 필수 레퍼런스

웹을 구성하는 3대 언어의 역할

HTML (뼈대 / Structure)



```
<p class="text">안녕하세요</p>
```

Basic Content Structure
(기본 콘텐츠 구조)

CSS (인테리어 / Design)



```
.text {  
  color: blue;  
  font-size: 20px;  
}
```

Styling & Visual Presentation
(스타일 및 시각적 표현)

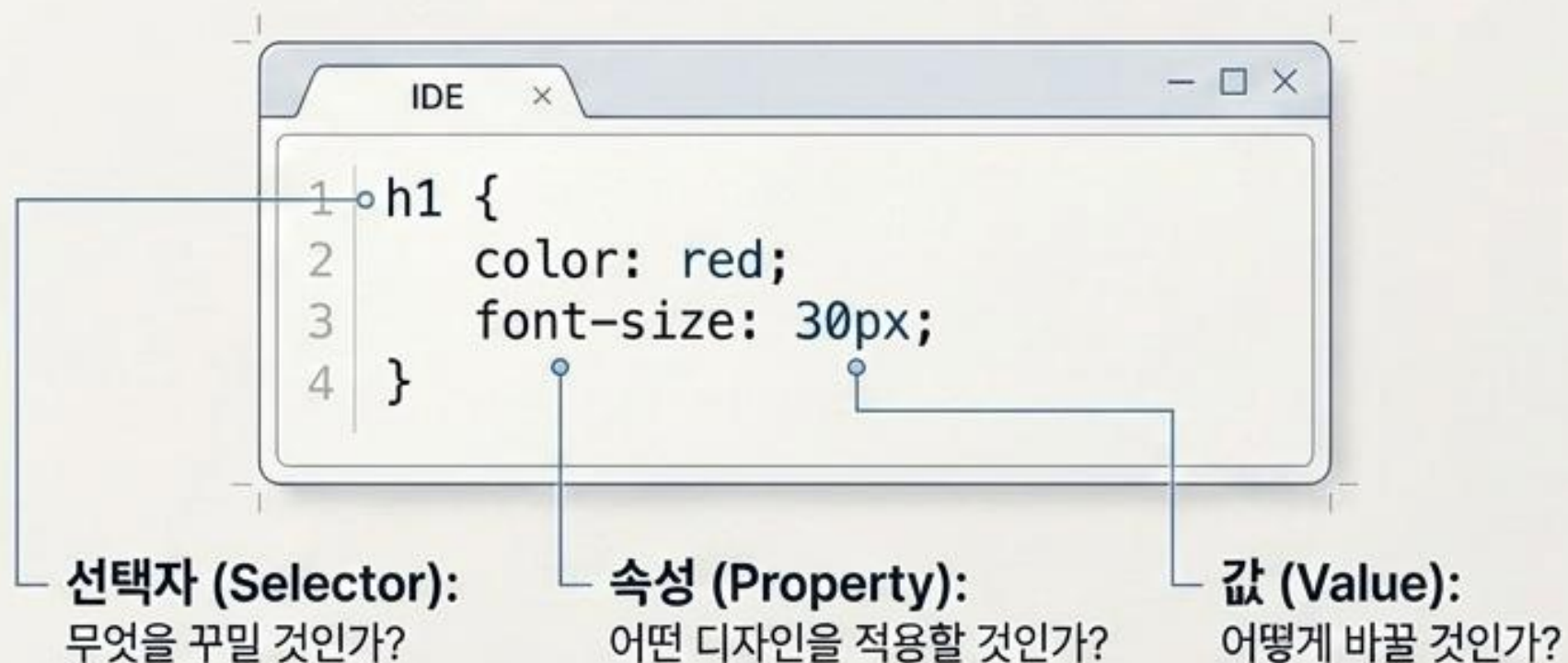
JavaScript (동작 / Function)



상호작용 및 동적 제어

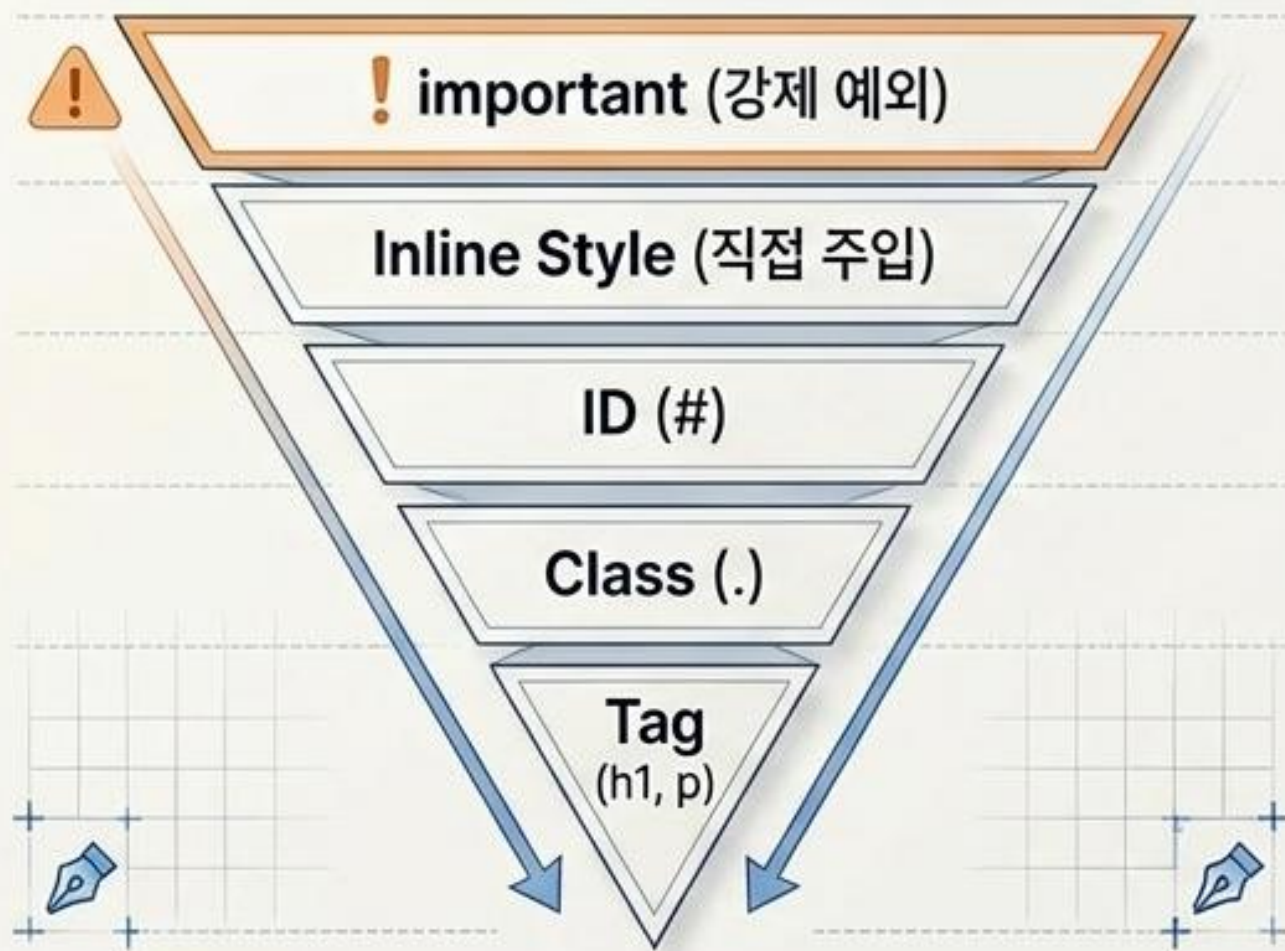
Interactive Behavior & Dynamic Control
(상호작용 및 동적 제어)

CSS 적용 방법과 기본 문법



권력의 피라미드

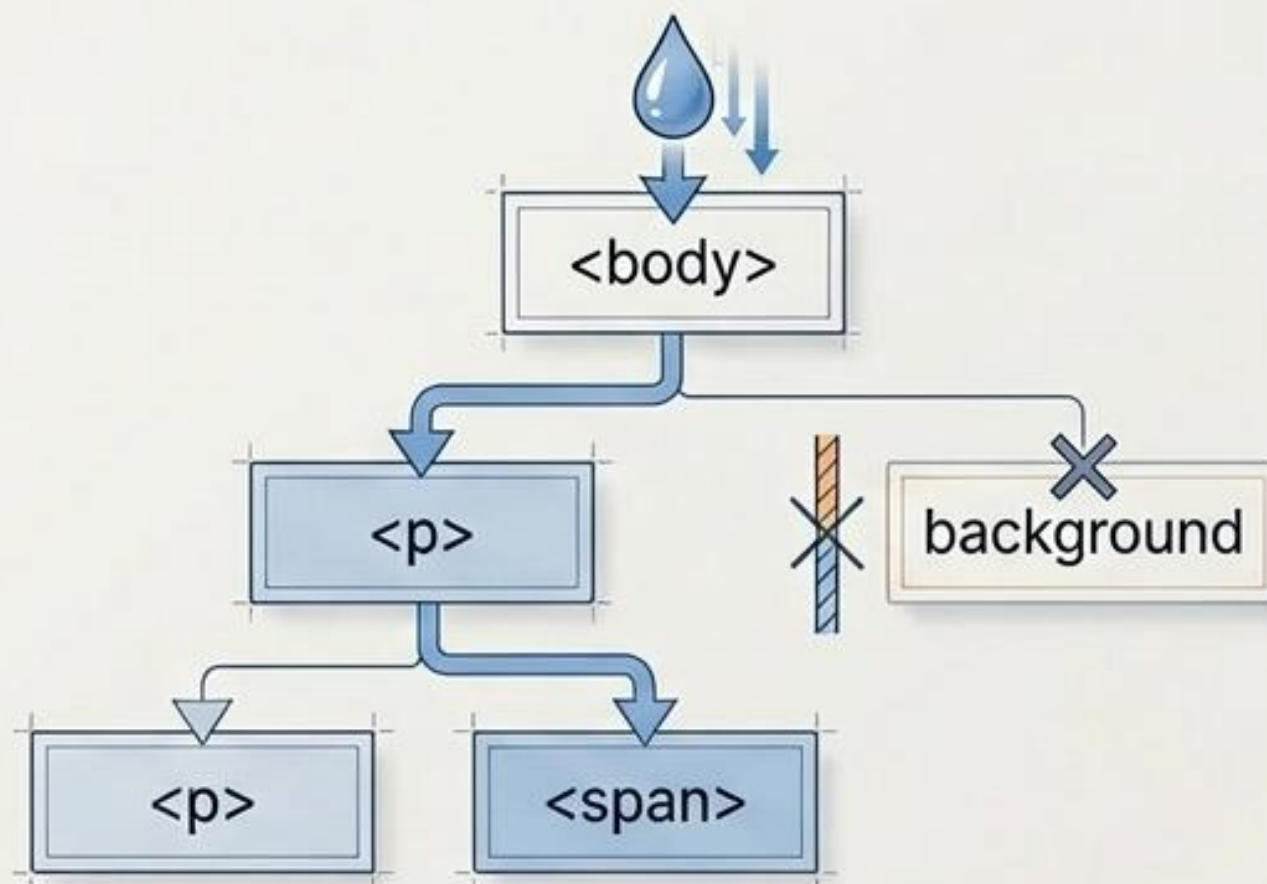
(Specificity & Cascading)



동일한 요소에 스타일이 충돌할 경우, 더 좁고 강력한 범위가 승리합니다. 가장 마지막에 선언된 코드도 영향을 미칩니다.

상속의 흐름

(Inheritance)



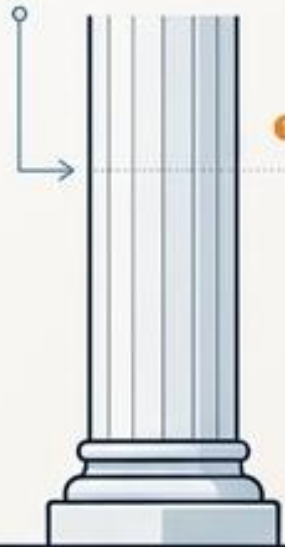
부모(body)의 color는 자식에게 흐르지만, background 같은 속성은 상속되지 않습니다.

CSS Masterclass

선택
(Selection)



Selectors

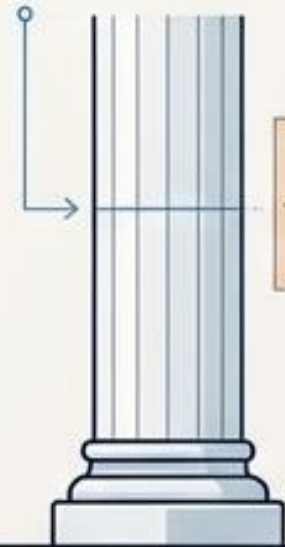


①
정확한 대상 지정
(Precise Targeting)

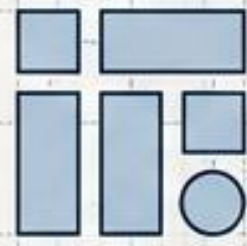
크기
(Sizing)



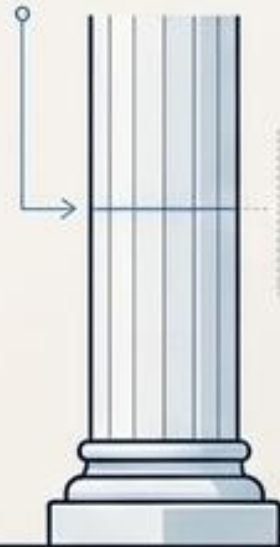
Box Model, Units



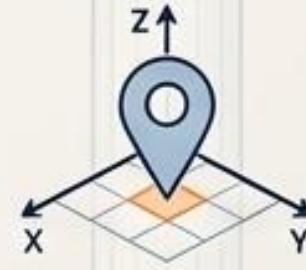
정렬
(Alignment)



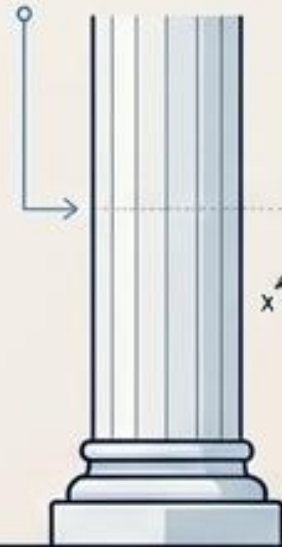
Flexbox, Grid



위치
(Positioning)



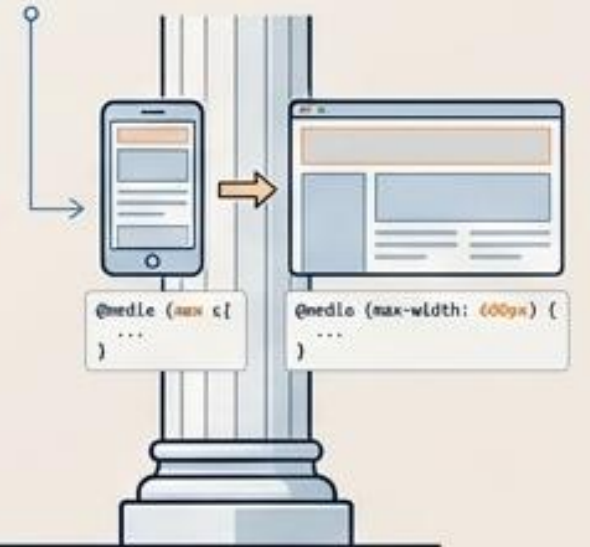
Position, Z-index



반응형
(Responsiveness)



Media Query, Animation



CSS는 결국 이 5가지의 조합입니다.



Pillar 1. 무엇을 꾸밀 것인가? (Selection)



HTML Input

```
<div id="hero">  
  <h1 class="title">Title</h1>  
  <p>Text</p>  
  <button>Click</button>  
</div>
```

CSS Selectors

- * (전체 선택자: 모든 요소 초기화)
- p (태그 선택자)
- .title (클래스 선택자: 가장 범용적, 재사용 가능)
- #hero (ID 선택자: 문서 내 단 한 번만 사용)
- div > p (자식/하위 선택자: 구조적 접근)

가상 클래스 & 요소 (Pseudo-classes & Elements)

button:hover

input:focus

p::before

p::after



Pillar 2. 얼마나 공간을 차지하는가? - 단위 (Units Guide)

| 단위 (Unit) | 유형 (Type) | 특징 (Characteristics) | 실무 추천 용도 (Best Practice) |
|-----------|-----------|-----------------------------------|------------------------------|
| px | 절대 단위 | 화면 크기에 상관없이 고정 | 정밀한 테두리(Border), 고정된 아이콘 |
| % | 상대 단위 | 부모 요소 기준 비율 | 유동적인 박스 너비 (max-width: 100%) |
| rem | 상대 단위 | 최상위 요소(html) 폰트 크기 기준 | 폰트 크기 (접근성 필수), 여백 |
| vw / vh | 상대 단위 | 브라우저 뷰포트(화면) 기준
(1vw = 너비의 1%) | 풀스크린 레이아웃, 반응형 타이포그래피 |

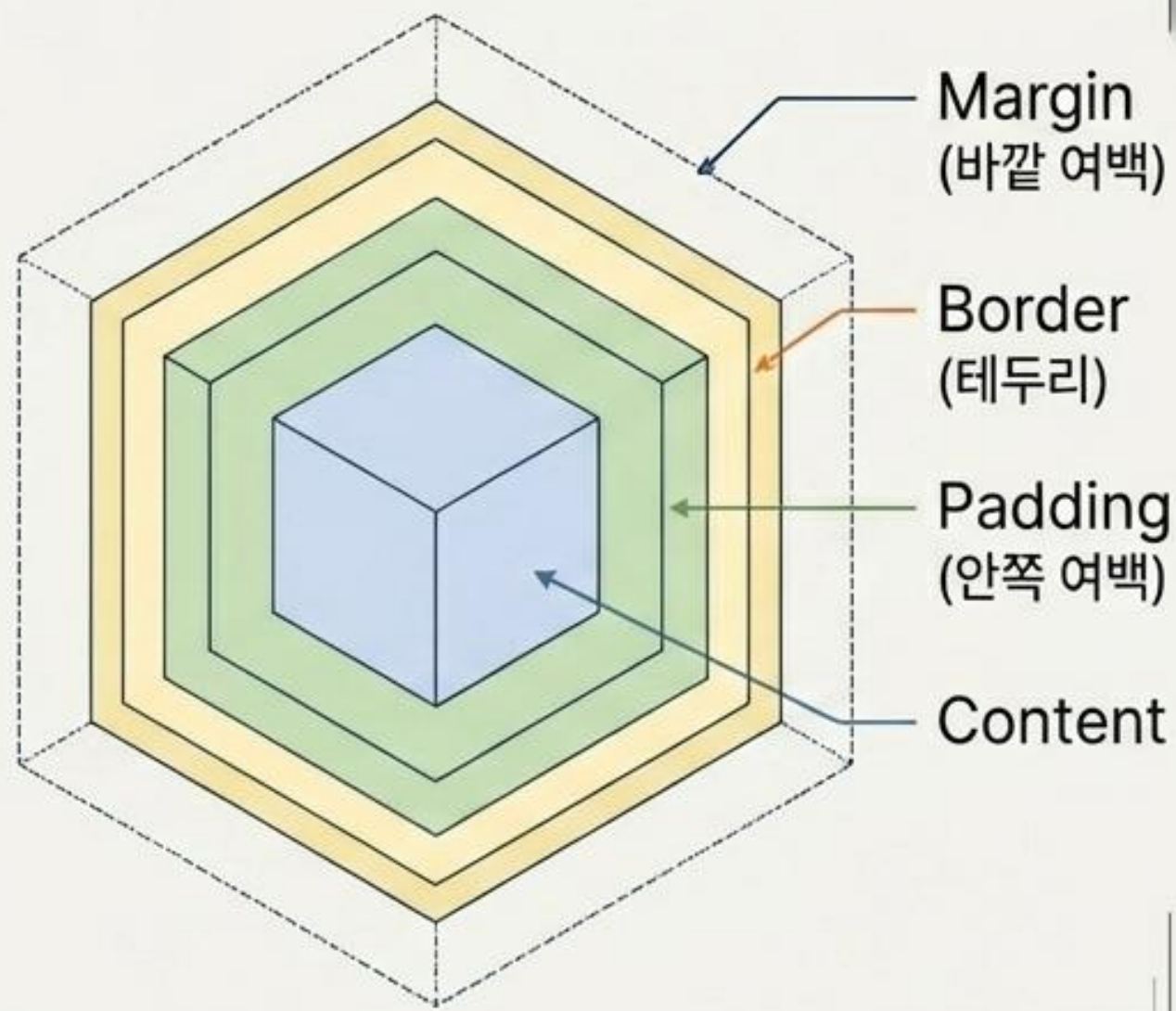


Pro-Tip Callout

실무 핵심: 폰트는 rem, 반응형 레이아웃은 vw/vh를 우선 고려하세요.

Pillar 2. 모든 것은 네모난 상자다 (The Box Model)

The Box Model Layers



The Math of box-sizing

Without border-box (기본값):

$$\begin{array}{l} \text{width} \\ 300\text{px} \end{array} + \begin{array}{l} \text{padding} \\ 20\text{px} \end{array} = \text{총 너비 } 340\text{px} \quad \text{(레이아웃 파괴)}$$

With box-sizing: border-box (실무 필수):

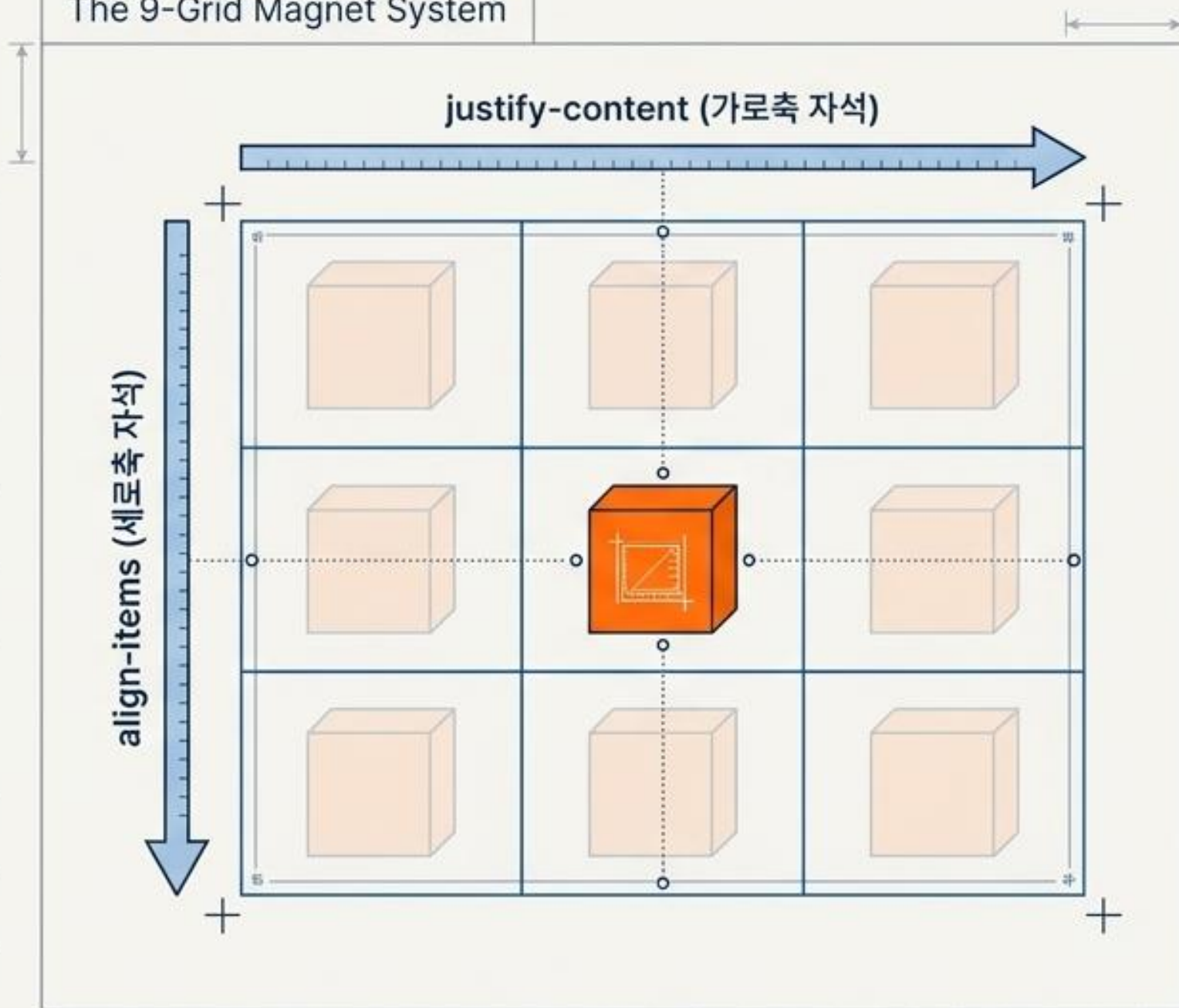
$$\begin{array}{l} \text{width} \\ 300\text{px} \end{array} \text{ (패딩과 테두리 포함)} = \text{총 너비 } 300\text{px 유지}$$

```
* { box-sizing: border-box; }
```


Pillar 3. 어떻게 나열할 것인가? - 정렬 마스터 키 (Flexbox)

Flexbox는 1차원(행 또는 열)을 지배합니다.

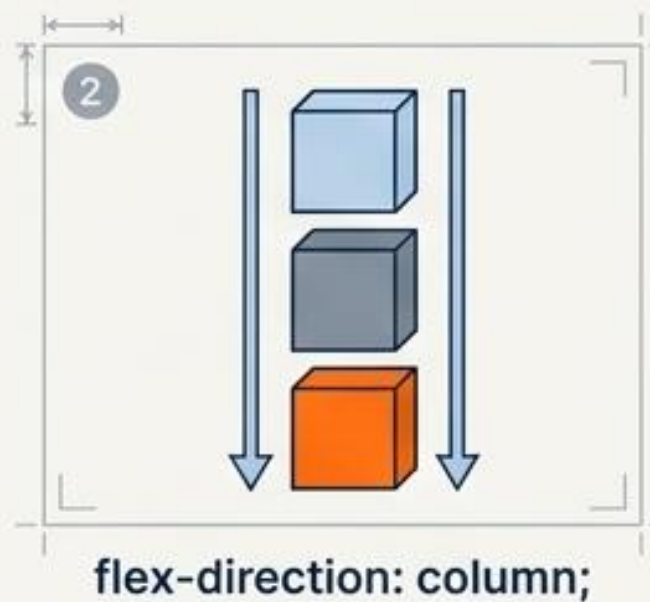
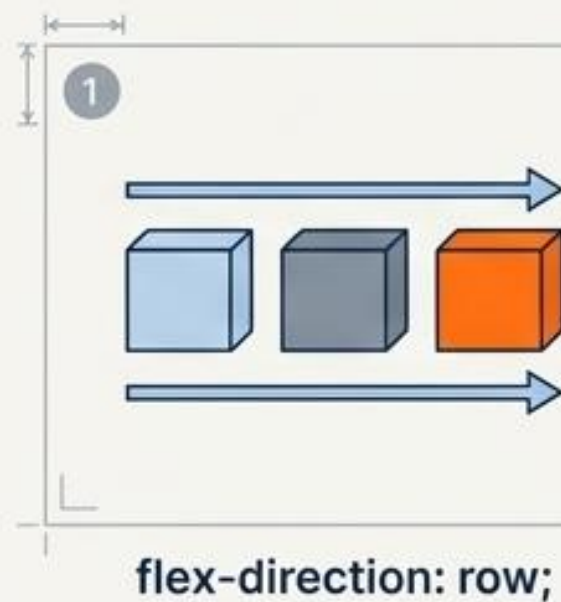
The 9-Grid Magnet System



The Golden Snippet & Controls

실전 중앙정렬 (웹 디자인에서 가장 많이 쓰이는 공식)

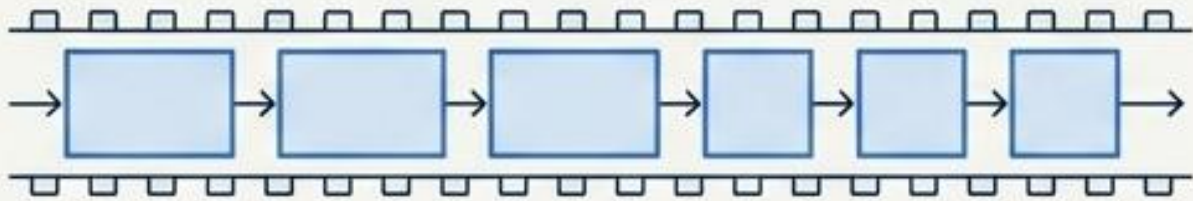
```
display: flex; — 플렉스 컨테이너 설정  
justify-content: center; — 가로축 중앙  
align-items: center; — 세로축 중앙
```



1차원 vs 2차원 레이아웃 (Flexbox vs Grid)

Flexbox (1차원 / 1D)

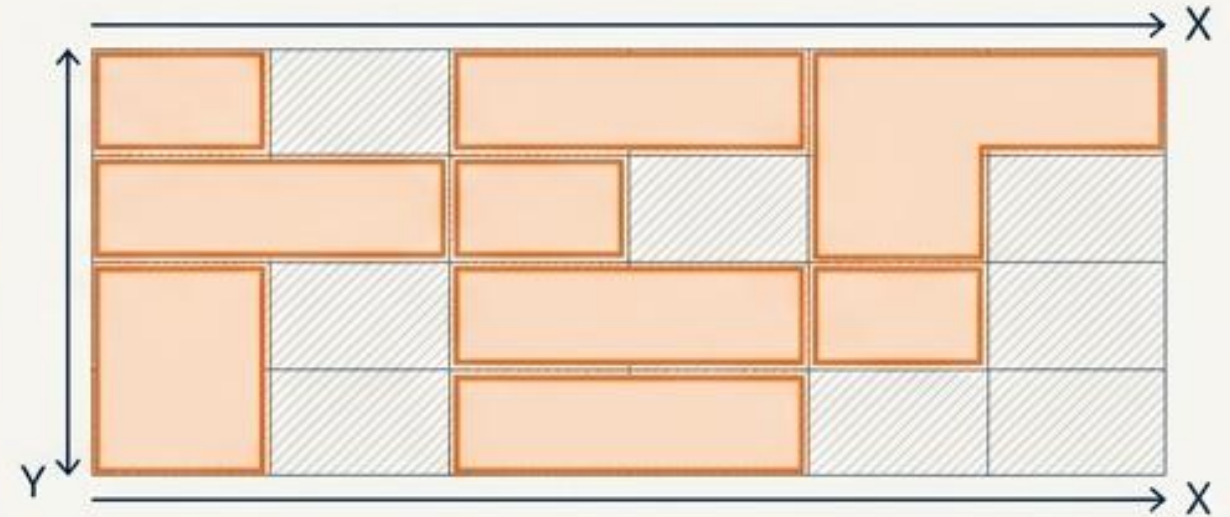
```
display: flex;
```



- 콘텐츠 중심 배열
- 행이나 열 중 하나의 방향으로 흐름
- UI 컴포넌트 내부 정렬에 최적화

Grid (2차원 / 2D)

```
display: grid;  
grid-template-columns: 1fr 1fr 1fr;
```



- 레이아웃 중심 배열
- 행과 열을 동시에 제어
- 전체 페이지의 골격을 잡을 때 최적화



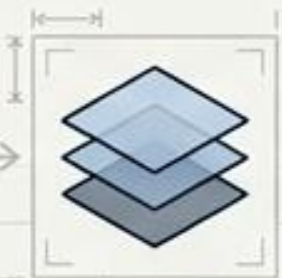
Decision Rule: 유동적인 내부 나열은 Flex, 전체 페이지의 바둑판 골격은 Grid를 선택하세요.



Pillar 4. 어디에 둘 것인가? (Positioning System)

| Position 속성 | 기준점 (Context) | 문서 흐름 이탈 (Leaves Flow) | 스크롤 반응 (Scroll) |
|--------------|-----------------------------------|------------------------|-----------------|
| static (기본값) | 원래 위치 | No | 스크롤 됨 |
| relative | 자신의 원래 위치 | No | 스크롤 됨 |
| absolute | 가장 가까운 Position 부모 | Yes
(공간 차지 안함) | 스크롤 됨 |
| fixed | 브라우저 창 (Viewport) | Yes | 화면에 고정 |
| sticky | 스크롤 전은 relative,
지점 도달 시 fixed | No | 조건부 고정 |

⚠ 주의:
항상 부모 요소에
relative를 부여하세요!



— Z-index 제어: `z-index: 999;`
(겹침 순서를 제어. 단, static 제외
position 속성이 적용되어야 동작함.)

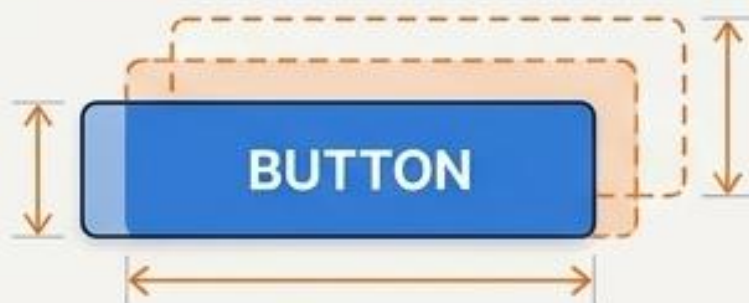
Pillar 5. 적응하고 움직여라 (Responsiveness & Polish)



```
@media (max-width: 768px) {  
  .box {  
    width: 100%;  
  }  
}
```

💡 모바일 우선(Mobile-First): min-width를 기본으로 짜고 max-width로 큰 화면을 제어하는 전략이 유리합니다.

Polish: Transition & Transform



```
transform: scale(1.2);  
transition: all 0.3s ease;
```

Modern CSS Sneak Peek



@keyframes (복잡한 애니메이션)



clamp() (유동적 폰트 크기)



@container (최신 컴포넌트 기반 반응형)

The Blueprint in Action (실전 카드 UI 예제)

Card Title

This is a beautiful, clean placeholder text for the UI card component. It demonstrates the visual output.

```
.card {  
  width: 300px;  
  padding: 20px;  
  border-radius: 12px;  
  background: white;  
  box-shadow: 0 4px 10px rgba(0,0,0,0.1);  
}  
  
.card:hover { transform: translateY(-5px); }
```

The Canvas - Output

The Blueprint - Input

실무자를 위한 CSS 생존 가이드 (Best Practices)

Architecture & Setup

☑ Reset CSS

```
* {  
  margin: 0;  
  padding: 0;  
  box-sizing: border-box;  
}
```

모든 브라우저의 기본 여백 초기화

☑ CSS 변수 (Variables)

```
:root {  
  --main-color: blue;  
}
```

Micro-copy: 색상, 테마 일괄 관리

☑ BEM 아키텍처 네이밍

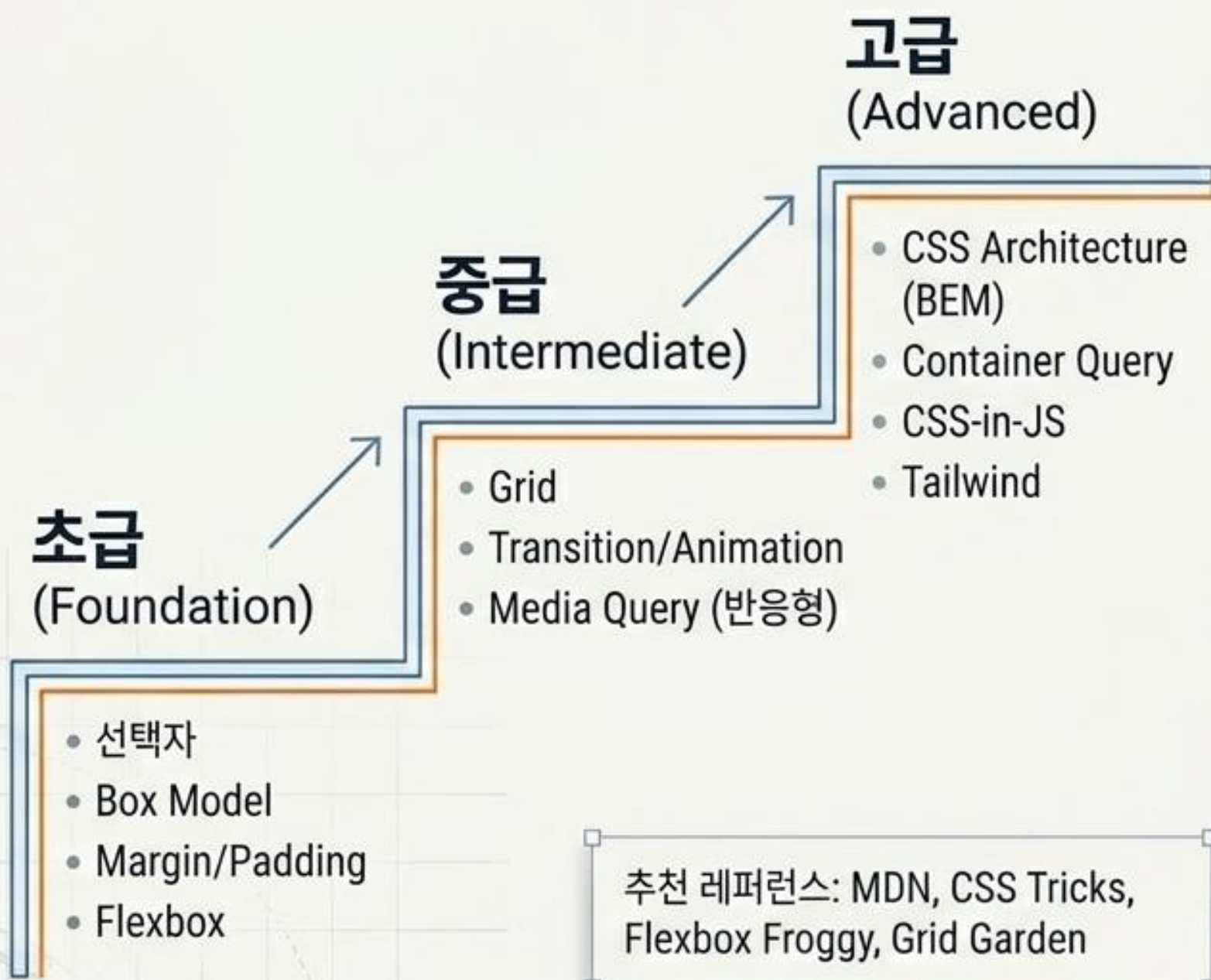
```
.card_title--active
```

Micro-copy: 블록(Block), 요소(Element), 상태(Modifier)를 명확히 구분

실무에서 가장 많이 쓰는 CSS TOP 10

```
display: flex; ← Layout & Alignment  
justify-content: center;  
align-items: center;  
gap: 20px;  
padding: 20px;  
margin: 0 auto;  
width: 100%;  
max-width: 1200px;  
border-radius: 12px; ← Elevation & Depth  
box-shadow: 0 4px 10px rgba(0,0,0,0.1);
```


다음 레벨을 위한 로드맵 (Learning Roadmap)



CSS는 결국:

1. 선택 (Selection)
2. 크기 (Sizing)
3. 정렬 (Alignment)
4. 위치 (Positioning)
5. 반응형 (Responsiveness)

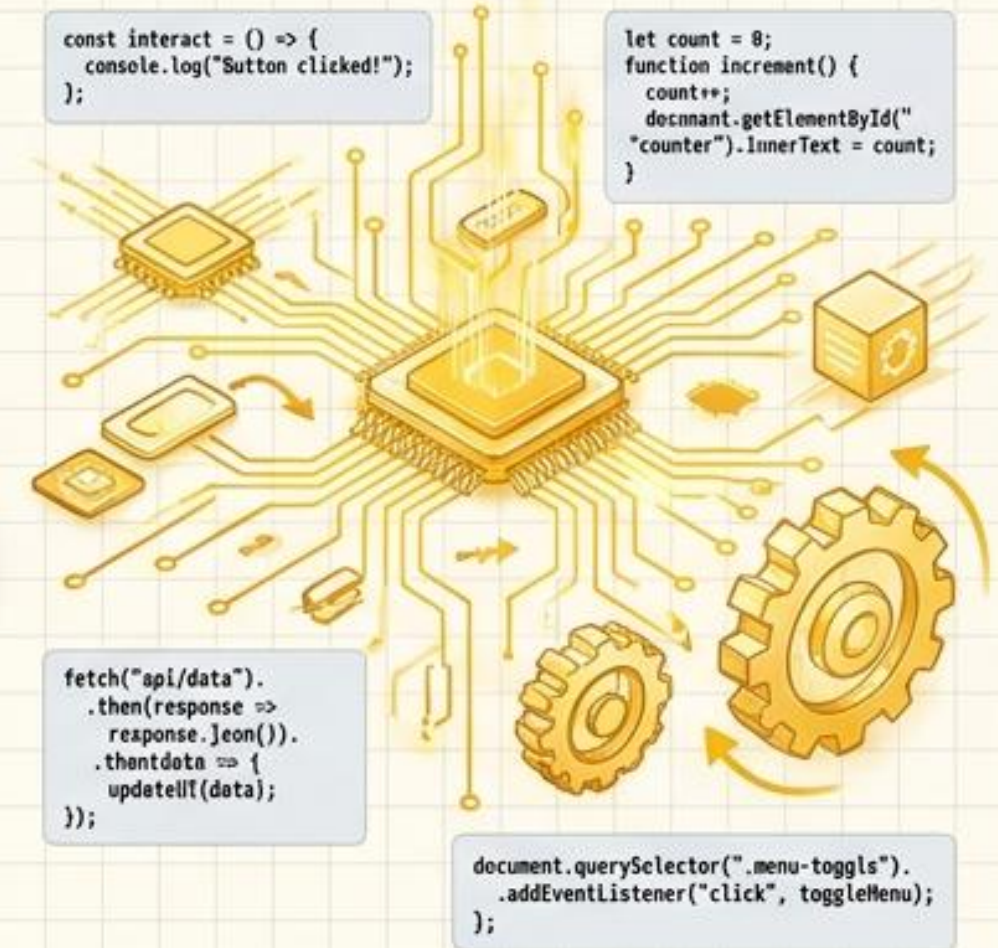
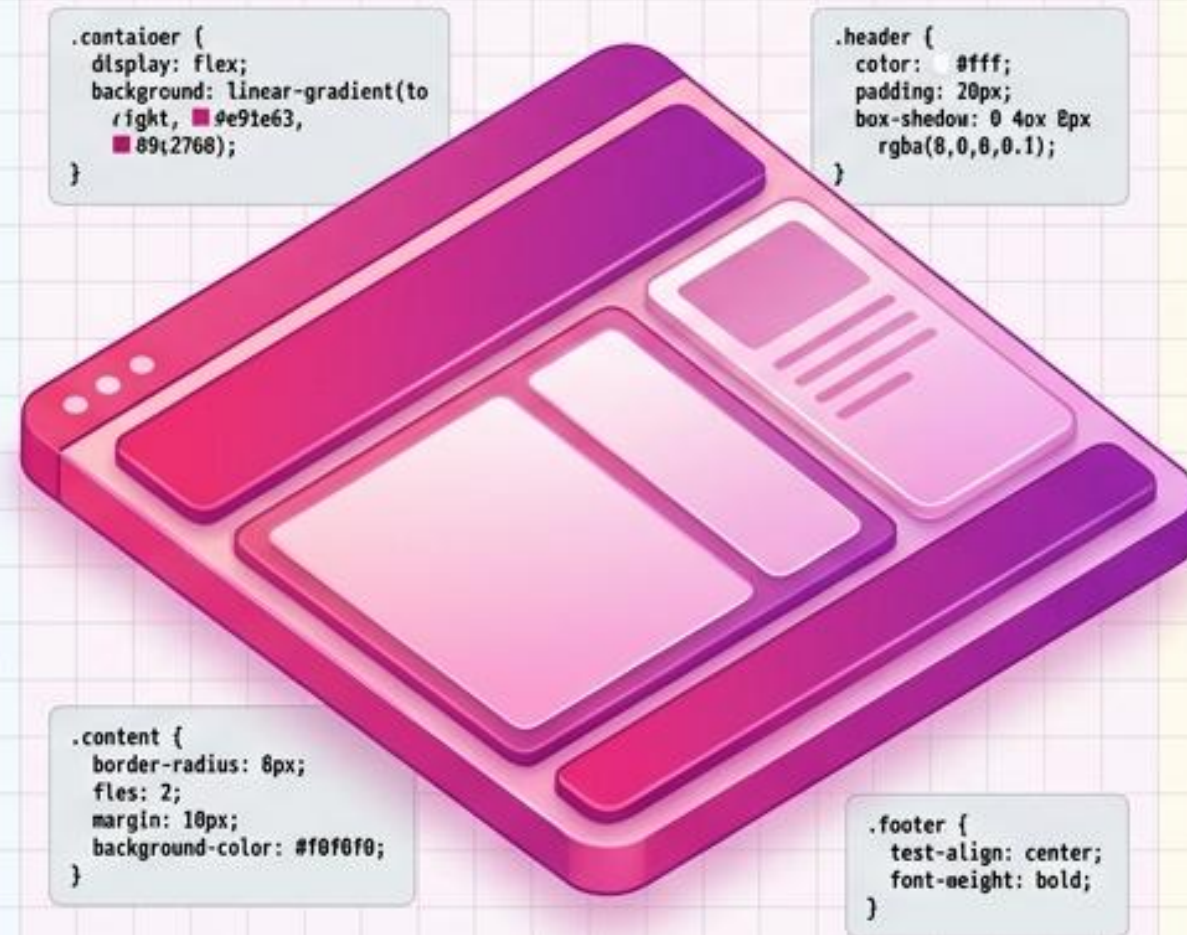
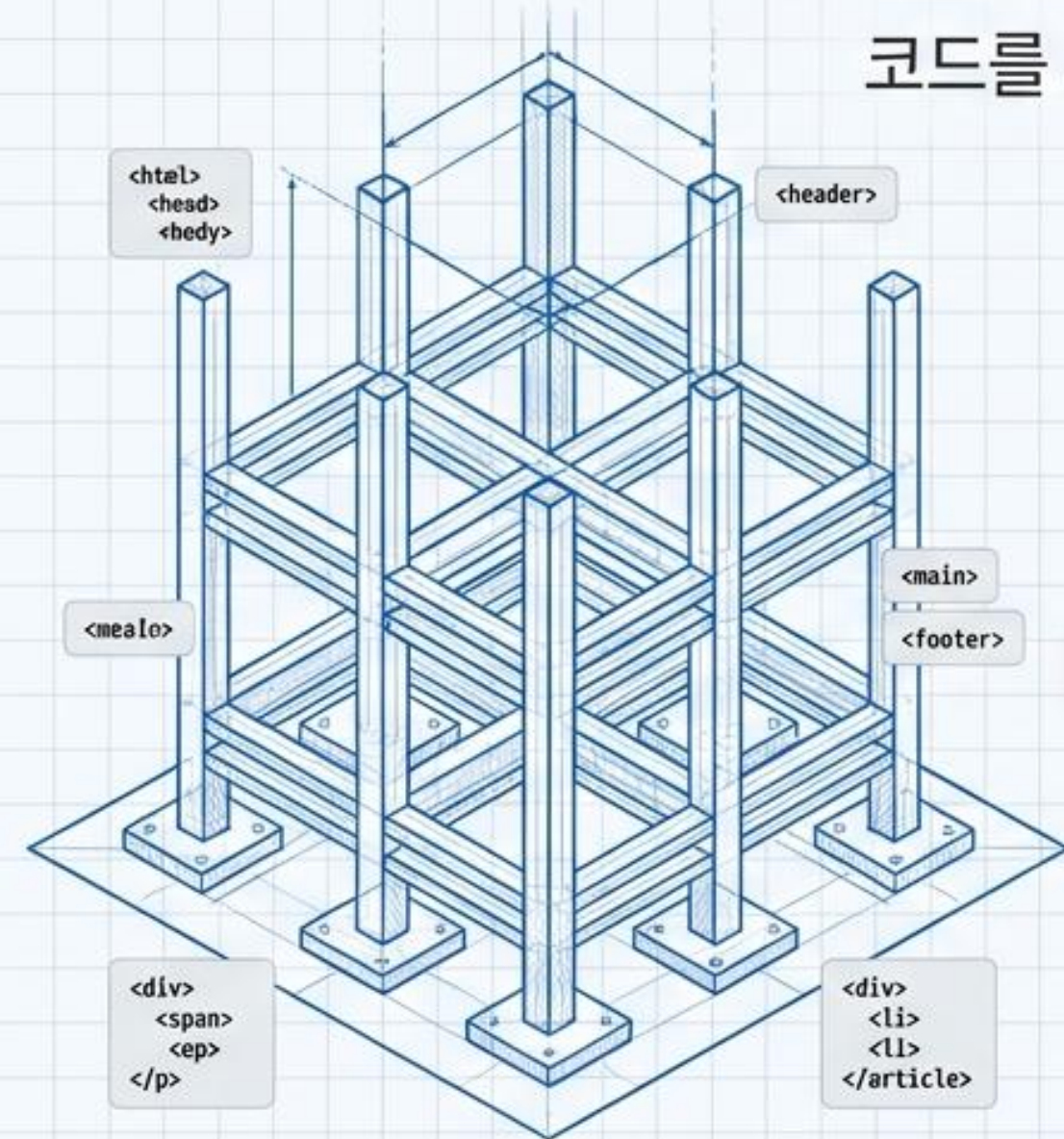
구조 (HTML)

표현 (CSS)

동작 (JavaScript)

웹을 만드는 3가지 언어

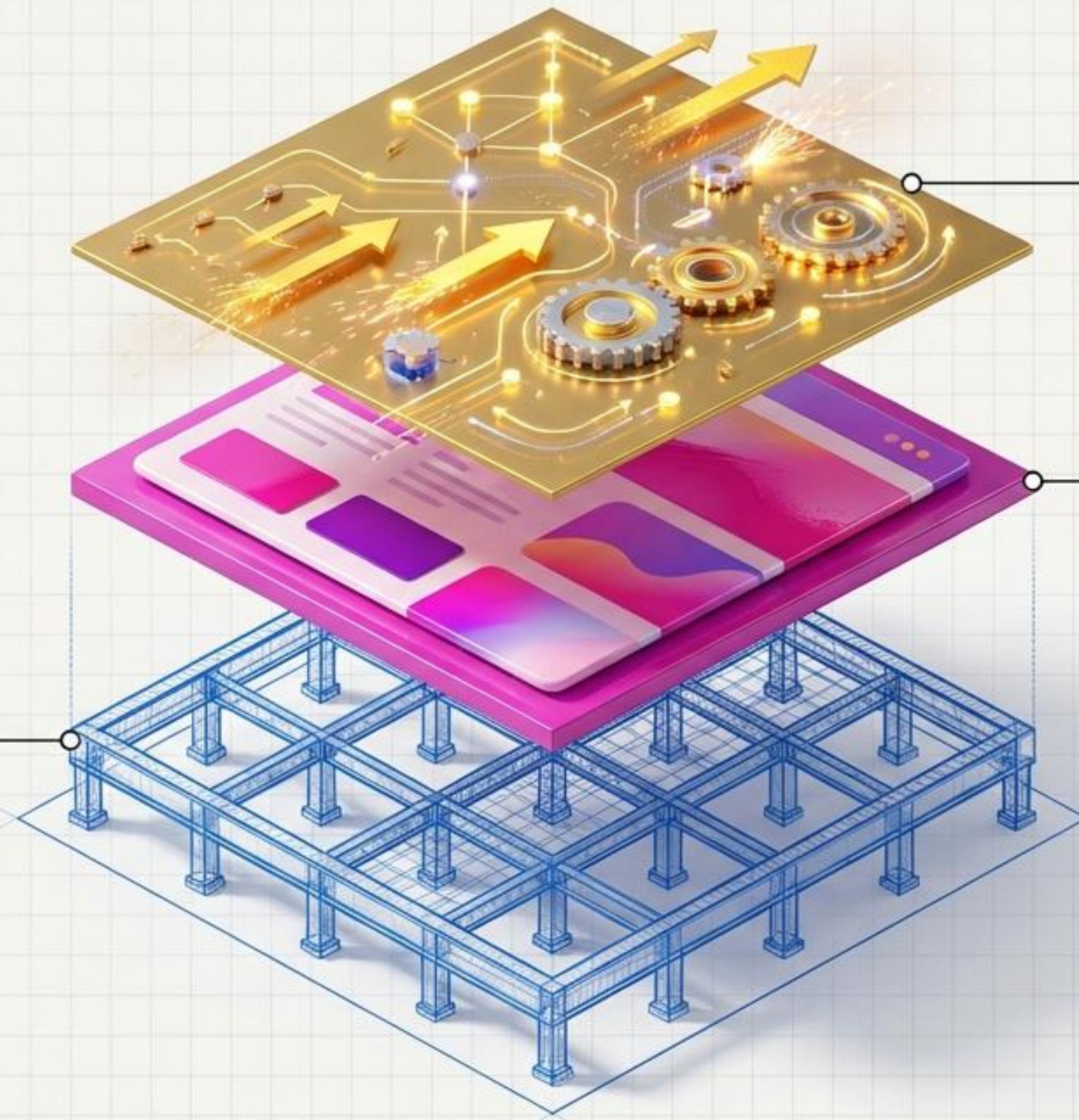
코드를 시각적으로 해독하는 프론트엔드 아키텍처 가이드



구조(HTML), 표현(CSS), 그리고 동작(JavaScript)의 완벽한 조화

HTML (구조와 정보)

뼈대 설계 - 브라우저와 검색엔진 (SEO)에게 문서의 성격을 설명하는 정보 마크업 언어.



JavaScript (생동감과 논리)

엔진 구동 - 변수와 제어 흐름을 통해 웹페이지에 생동감을 불어넣는 프로그래밍 언어.

CSS (디자인과 공간)

시각적 표현 - 정보의 배치, 색상, 여백을 결정하고 시각적 가독성을 부여하는 스타일 시트.

HTML의 올바른 사용: 구조적 마크업 vs 단순한 흉내

WRONG WAY



```
<div style='font-size: 30px; margin: 20px;'>아이유 팬명록</div>
```



단순한 시각적 흉내 (단순히 폰트를 키운 글)

RIGHT WAY



```
<h1>아이유 팬명록</h1>
```



구조적 마크업 (브라우저가 '제목'으로 인식하는 글)

HyperText Markup Language & SEO: HTML은 브라우저와 소통하는 언어입니다.
태그를 목적에 맞게 사용해야 검색엔진이 내 웹사이트를 올바르게 읽고 분류할 수 있습니다(검색 최적화, SEO).



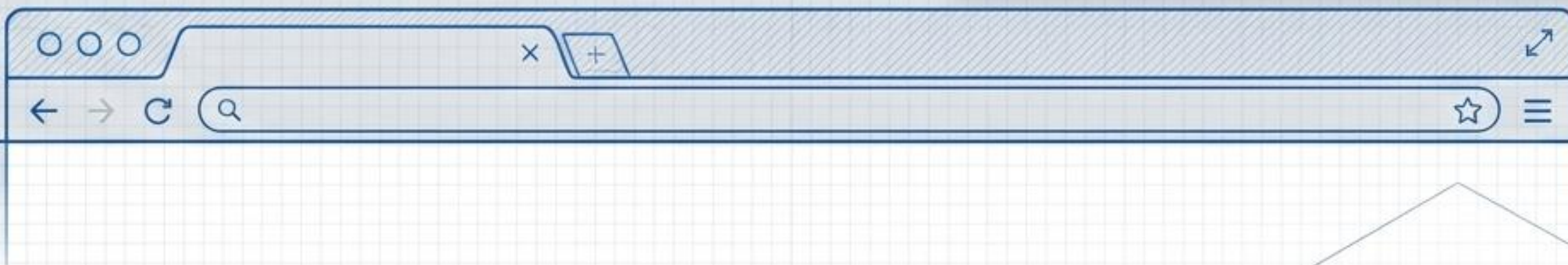
<head> (문서의 뇌 / 보이지 않는 정보)

웹브라우저가 문서를 읽을 때 필요한 설정값과 요약 정보.

<meta> : 문서의 초기 정보 및 검색엔진 요약

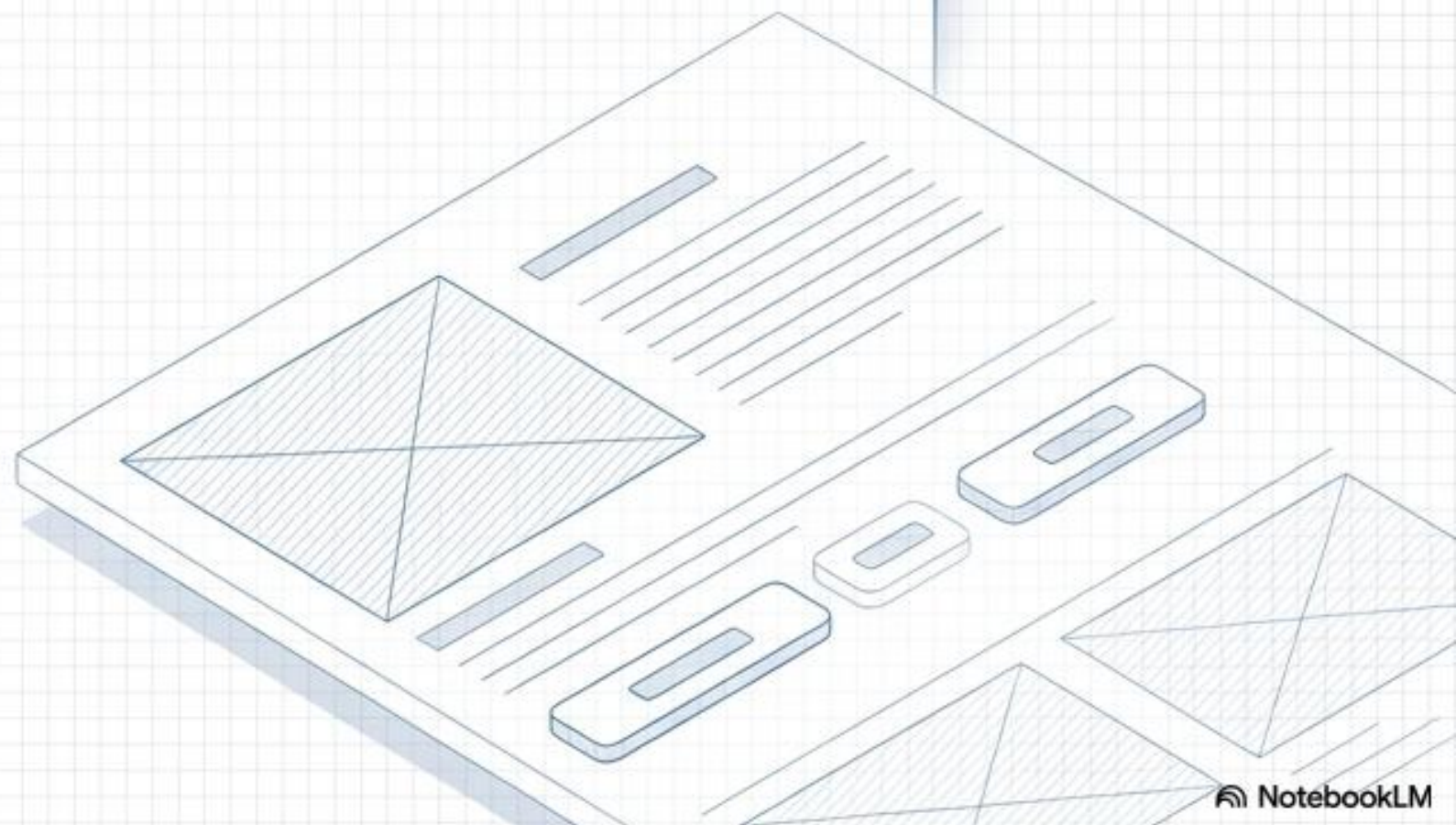
<style> : CSS 문법을 적용하는 공간

<script> : JavaScript 등 스크립트를 적용하는 공간



<body> (문서의 몸 / 보여지는 콘텐츠)

웹브라우저 내부에 실제로 표시되는 모든 정보와 마크업 콘텐츠. (예: <h1>, <nav>)



HTML 태그 구조 분석 (HTML Tag Structure Analysis)

여는 기호
(태그의 시작)

태그 이름
(컨텐츠의 성격 정의)

속성(Property)
(태그에 추가적인 정보나 식별자 부여)

```
</h1 class='example'>아이유 팬명록</h1>
```

닫는 기호

컨텐츠
(실제 화면에 표시될 내용)

닫는 태그
(슬래시 /와 함께 사용하여
해당 태그의 범위가 끝났음을
브라우저에 알림)

Digital Architecture and Isometric Blueprint 디자인과 캐스케이딩 (The Cascading Waterfall)

HTML의 [데이터 구조] 목적을
훼손하지 않기 위해 분리되어 탄생한
디자인 전용 문법입니다.
Cascading은 위에서 아래로 흐르는
폭포처럼, 상위 속성이 하위로
상속(부모-자식 구조)되며 중첩 시 엄격한
우선순위를 따르는 방식을 뜻합니다.

Cascading은 위에서 아래로 흐르는 폭포처럼,
아래 속성이 상위 속성이 하위로 상속(부모-자식 구조)되며
중첩 시 엄격한 우선순위를 따르는 방식을 뜻합니다.



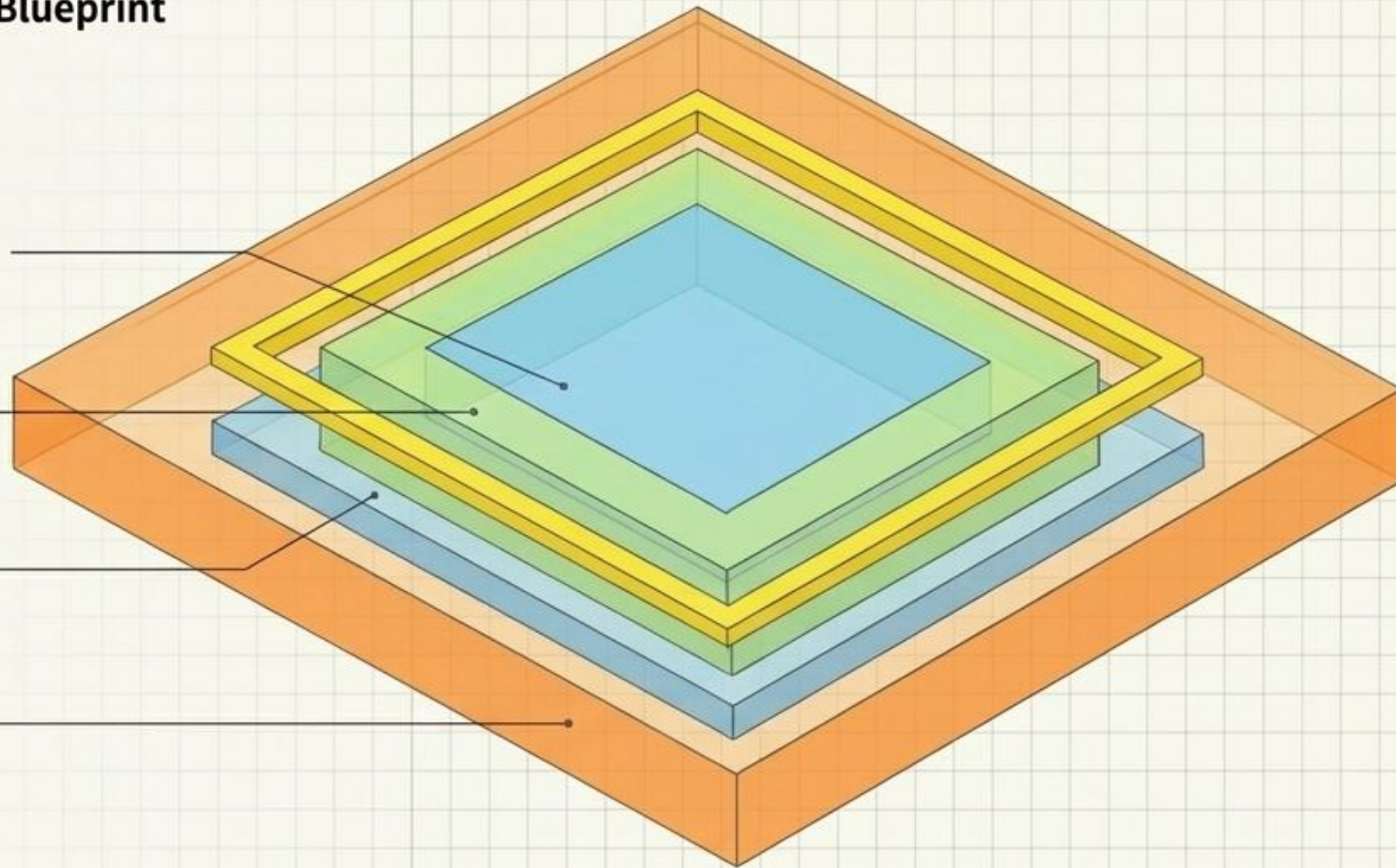
CSS Box Model

Content (하늘색): 요소의 실제 내용물 (width, height로 조절)

Padding (연두색): 내부 여백 (콘텐츠를 감싸는 보호 영역)

Border (노란색): 테두리 영역 (실선, 점선 등 지정 가능)

Margin (주황색): 외부 여백 (다른 요소와의 거리)



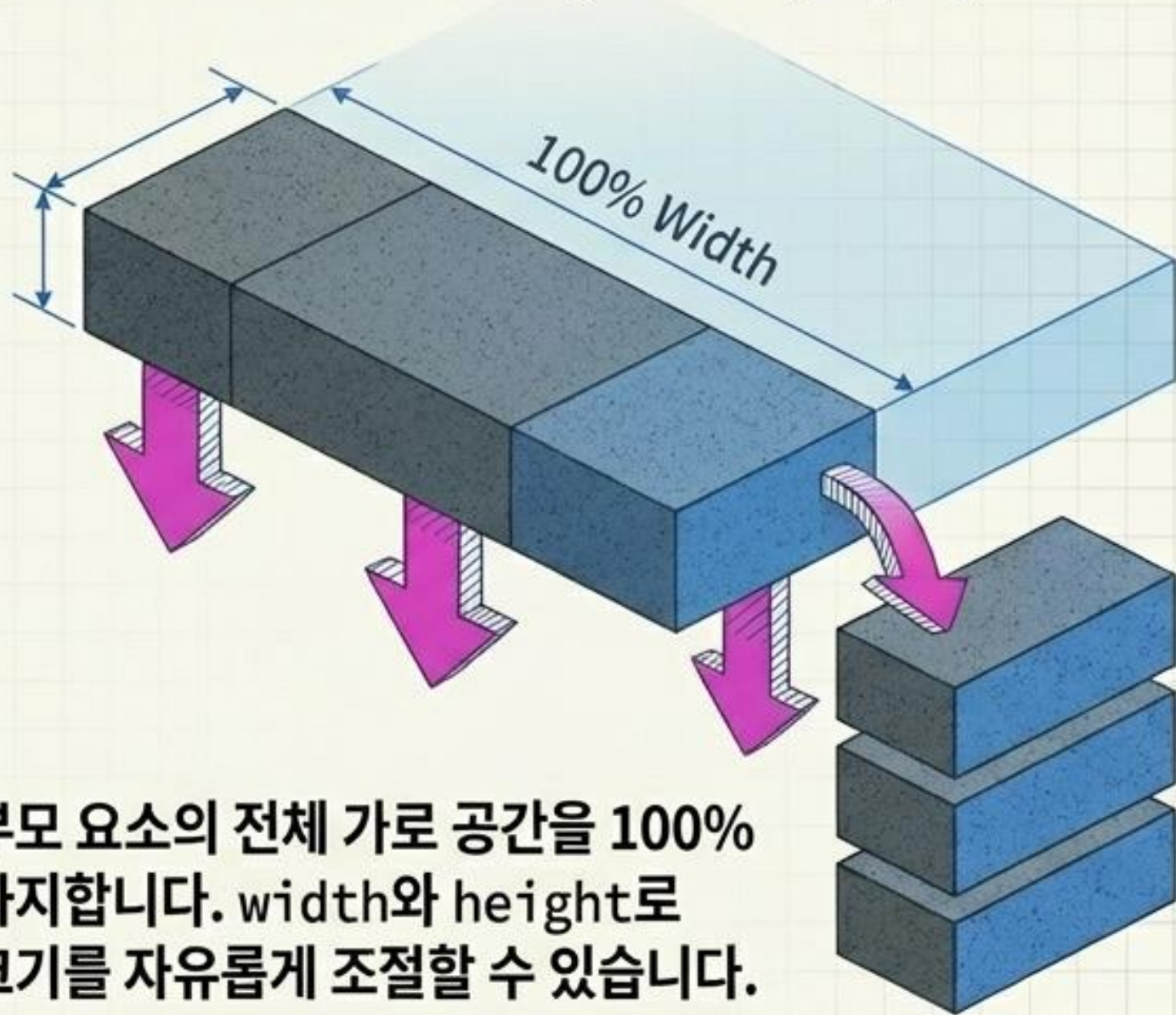
상하여백 중첩 (Margin Collapse)

인접한 두 요소의 상하 Margin이 다를 경우, 더 큰 값 하나로 중첩되어 적용됩니다 ($20\text{px} + 20\text{px} = 40\text{px}$ 이 아닌 20px).

box-sizing 규칙

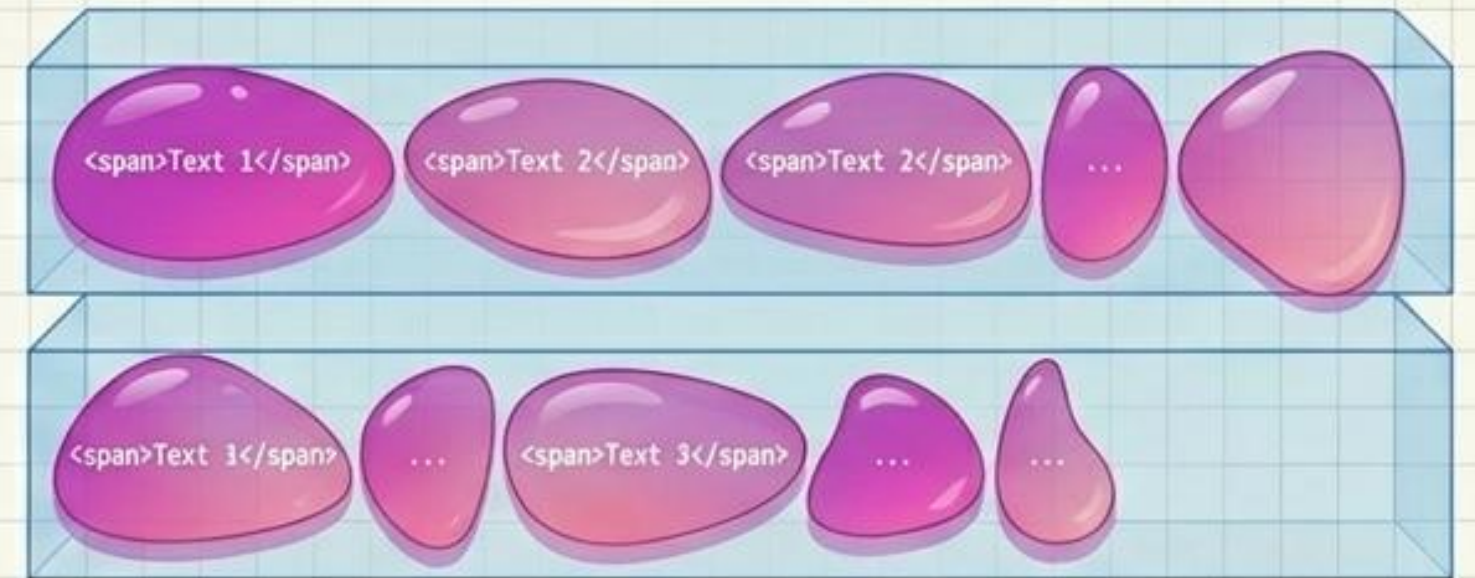
기본값 content-box는 콘텐츠 영역만 너비로 계산하지만, border-box를 사용하면 테두리(Border)까지 포함하여 요소의 총 크기를 직관적으로 제어할 수 있습니다.

Digital Architecture and Isometric Blueprint Block Elements (<div>, <p>)



부모 요소의 전체 가로 공간을 100% 차지합니다. width와 height로 크기를 자유롭게 조절할 수 있습니다.

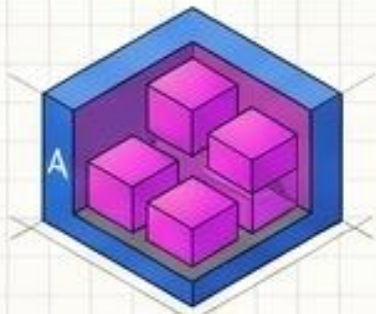
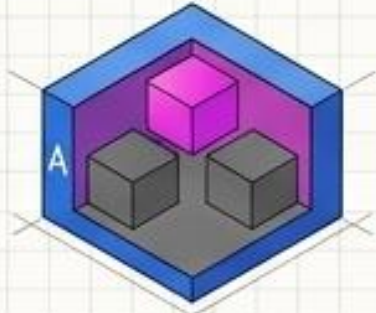
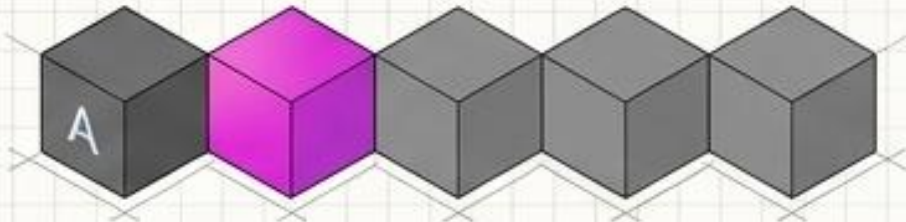
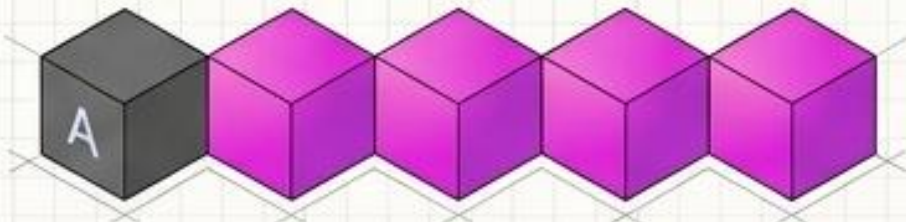
Digital Architecture and Isometric Blueprint Inline Elements ()



딱 자신의 콘텐츠 크기만큼만 영역을 차지합니다. 가로로 나란히 배치되며, 기본적으로 width와 height 값이 적용되지 않습니다.

`display: inline-block` 을 사용하면 나란히 배치 배치되면서도 블록처럼 크기를 조절할 수 있습니다.

Digital Architecture and Isometric Blueprint: CSS Combinator Selectors (Periodic Table)

| 선택자 이름 | 기호 및 문법 | 시각적 타겟팅 |
|-----------------------------|---------------------|---|
| 하위 선택자
(Descendant) | <div>A B</div> |  |
| 자식 선택자
(Child) | <div>A > B</div> |  |
| 인접 형제
(Adjacent Sibling) | <div>A + B</div> |  |
| 일반 형제
(General Sibling) | <div>A ~ B</div> |  |

Digital Architecture and Isometric Blueprint

속성 선택자 (Attributes)

- `[속성=값]` : 정확히 일치 (예: button)
- `[속성~=값]` : 단어가 포함됨 (띄어쓰기 기준)
- `[속성|=값]` : 정확히 일치하거나 하이픈(-)으로 시작
- `[속성^=값]` : 해당 값으로 시작하는 요소
- `[속성$=값]` : 해당 값으로 끝나는 요소
- `[속성*=값]` : 어디든 포함된 요소

상태 및 구조 (Pseudo-States)

`:link` → `:visited` → `:hover` → `:active`

구조:

`:first-child`
`:nth-child(n)`
`:last-child`

가상 요소 (DOM에 없는 콘텐츠 생성):

`::before` (요소 앞)
`::after` (요소 뒤)

웹의 심장, 자바스크립트 (The Engine: JavaScript Variables)

웹페이지에 생동감을 불어넣는 브라우저 표준 프로그래밍 언어입니다.
변수(Variable)에 특정 데이터를 담고 동작을 정의합니다.

let (변할 수 있는 상자)

```
let a = 1;  
a = 2;
```



값이 변경될 수 있는 경우 사용합니다. (예: for 반복문에서 값이 계속 바뀌는 i)

const (잠긴 금고)

```
const first = 'Bob';
```



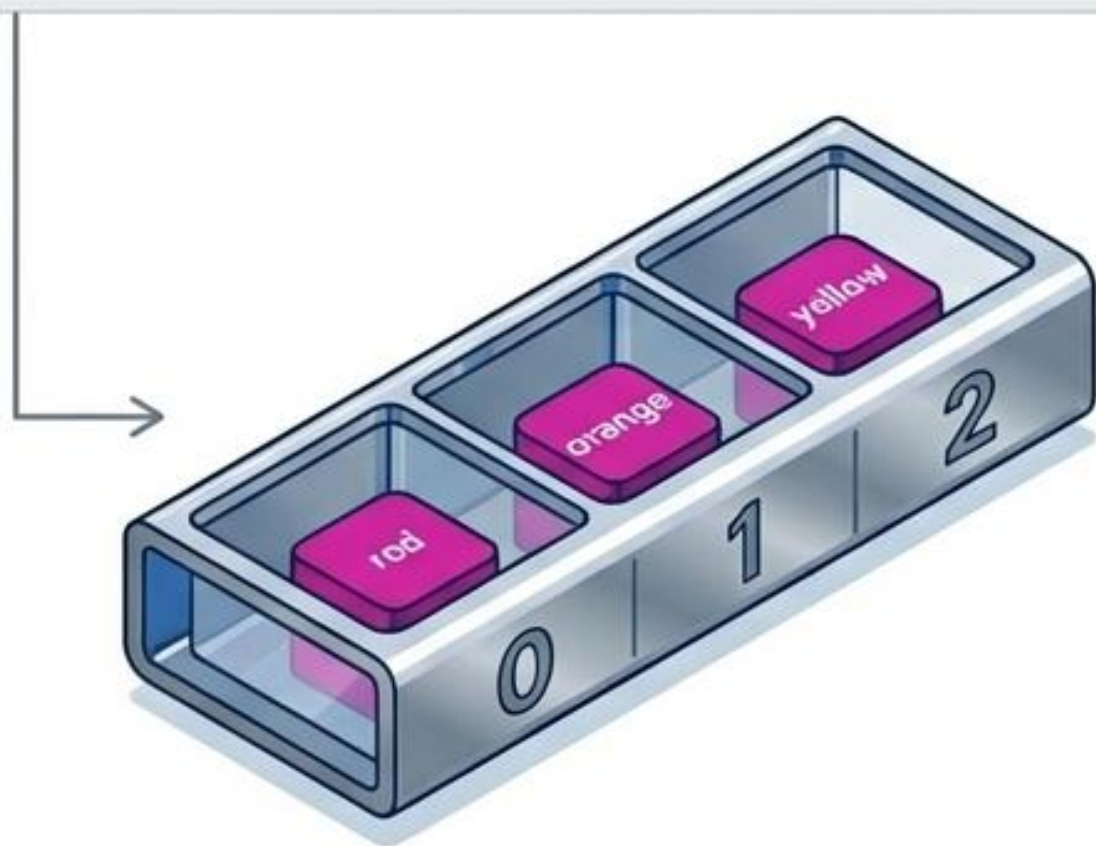
값이 절대 변경되지 않는 고정된 데이터를 선언할 때 사용합니다.

⚡ 문자열 더하기! 백틱(`)을 사용하거나 + 기호를 쓰면 문자가 연결됩니다. 숫자와 문자를 더하면 숫자가 문자로 강제 변환됩니다 ('Bob' + 1 = 'Bob1').

데이터 보관함: 배열(Array) vs 객체(Object)

배열 / 리스트 (Array)

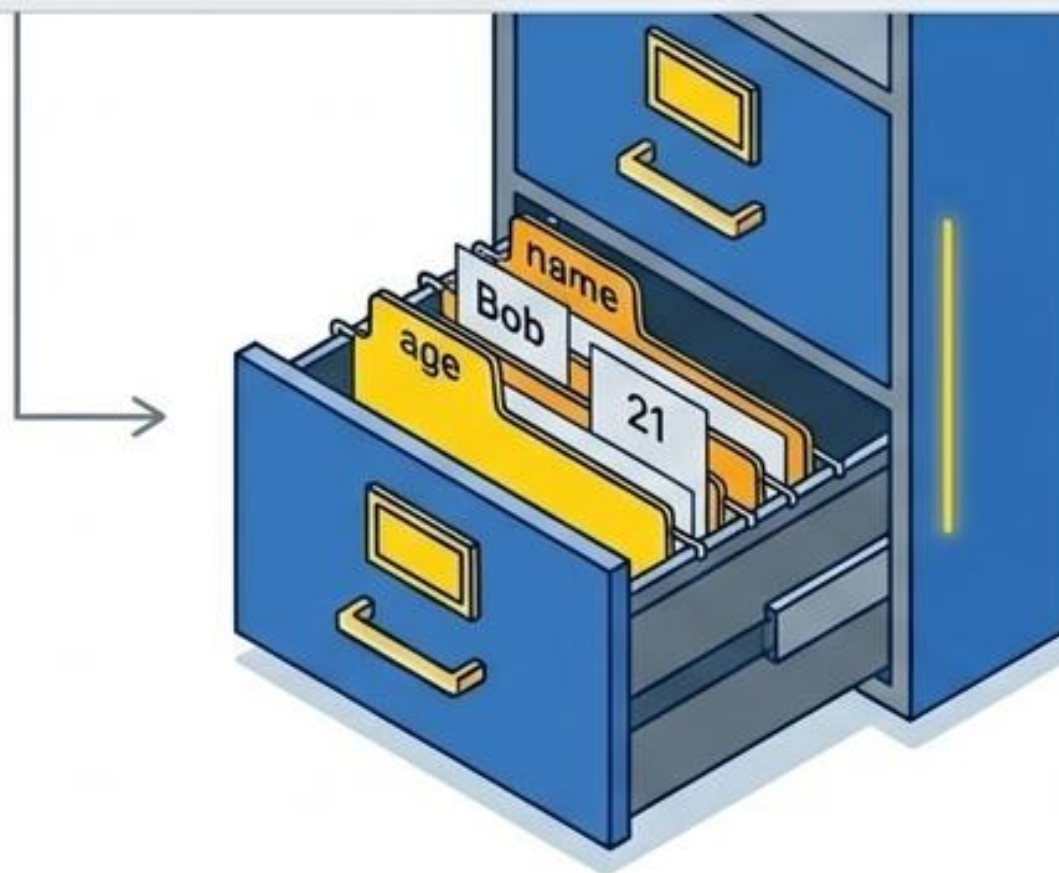
```
const rainbow = ['red', 'orange', 'yellow']
```



순서(Index)를 가진 데이터의 집합. 인덱스는 0부터 자동 부여됩니다. (rainbow[0]은 'red')

객체 / 딕셔너리 (Object)

```
let dict = {'name': 'Bob', 'age': 21}
```



사용자가 직접 이름(Key)을 지정하여 데이터를 관리. 순서가 존재하지 않습니다. dict['name']을 호출하면 'Bob'을 출력합니다.

함수의 작동 원리: 재사용 가능한 코드 공장 (How Functions Work)

Input (매개변수 / Parameter)



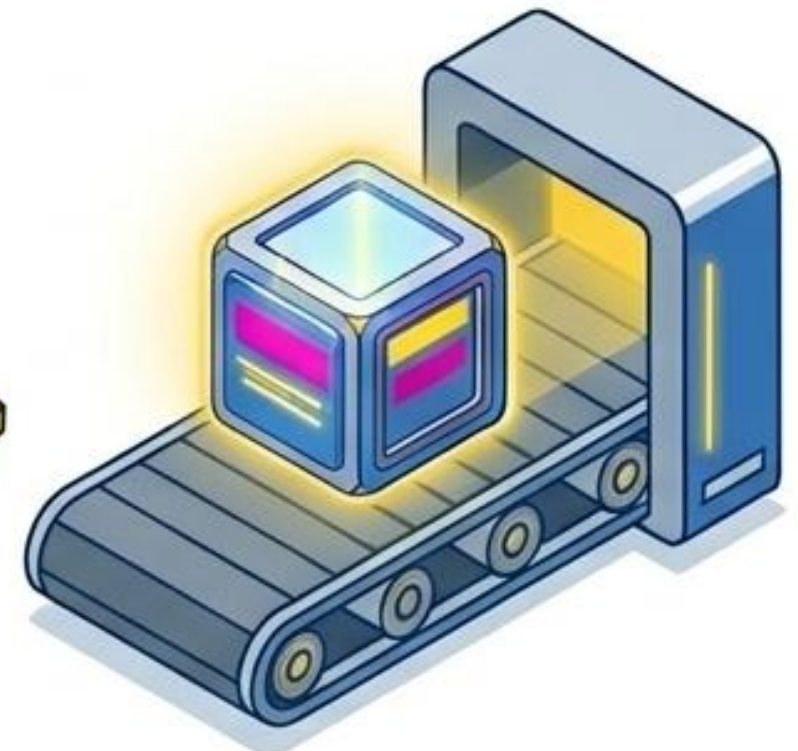
```
function calculateAvg(price1, price2)  
{
```

Process (실행 로직)



```
const sum = price1 + price2;  
const avg = sum / 2;
```

Output (Return 값)

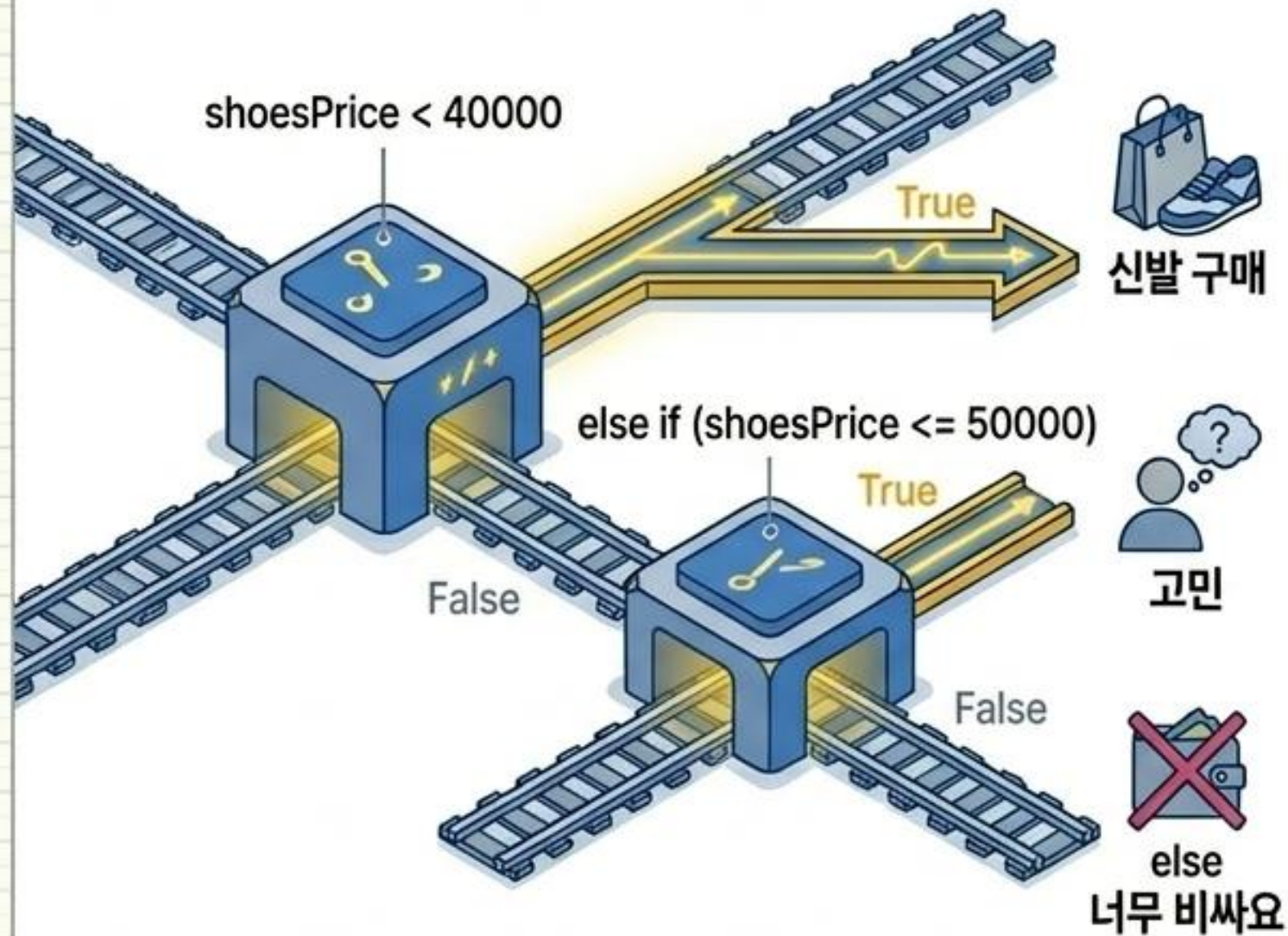


```
return avg;
```

⚡ 함수를 호출(`calculateAvg(1000, 2000)`)하면 공장이 가동되어 최종 결과물(1500)을 반환합니다. 동일한 코드를 여러 번 쓸 필요 없이 재사용하기 위해 사용합니다.

JavaScript 제어 흐름: 기계적 회로 동작 (Control Flow)

조건문 (If / Else If / Else)

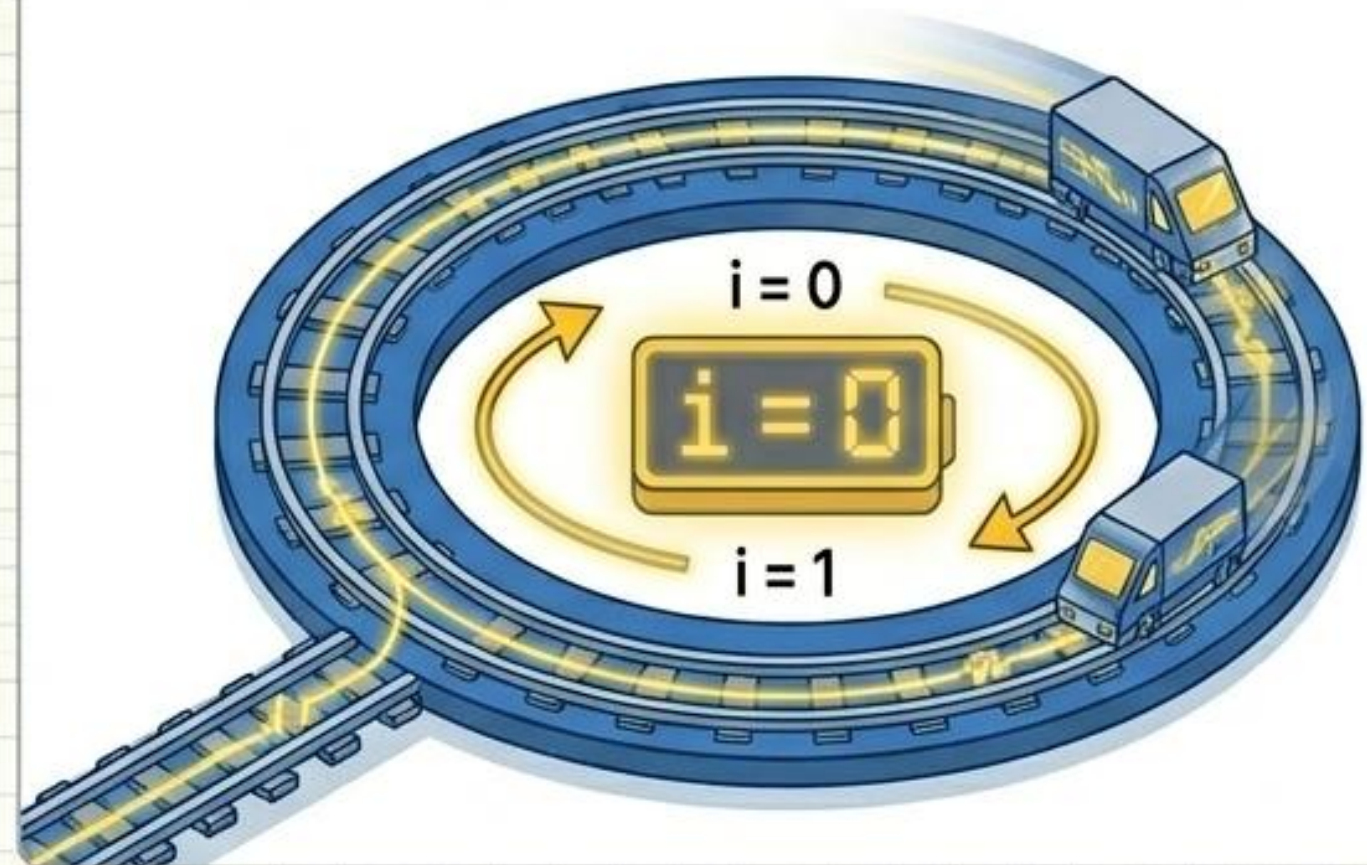


특정 로직이 True일 때만 코드를 실행하여 흐름을 제어합니다.

반복문 (For)

```
for (let i = 0 ; i < people.length ; i++)
```

1. 시작 조건
(let i = 0)
2. 실행 조건
(i < length - 배열의 길이까지만)
3. 실행 순서/증감
(i++ - 1씩 증가)



공장 가동 (Function):
show_gus(40) 호출 시
index 기준값을 40으로 설정.

트랙 반복 (For Loop):
mise_list 배열의 모든 데이
터를 처음부터 끝까지 순회.

논리 게이트 (If):
추출한 딕셔너리의 키값
['IDEX_MVL']이 기준값보다
작을 때만 문을 엽니다.

```
function show_gus(index) {  
  for (let i = 0; i < mise_list.length; i++) {  
    let mise = mise_list[i];  
    if (mise['IDEX_MVL'] < index) {  
      console.log(mise['MSRSTE_NM'], mise['IDEX_MVL']);  
    }  
  }  
}
```

데이터 추출 (Dictionary):
i번째 데이터를 꺼내 mise라는
변수(상자)에 임시 저장.

결과 출력:
조건에 맞는 지역 이름과 미세먼지
수치를 콘솔에 출력합니다!

축하합니다. 당신은 이제 **웹의 구조(HTML)**와 **공간(CSS)**을 넘어, **데이터의 흐름(JS)**을 해독할 수 있습니다.